Mattermost Cross Guard

Cross-domain message relay for Mattermost Federal

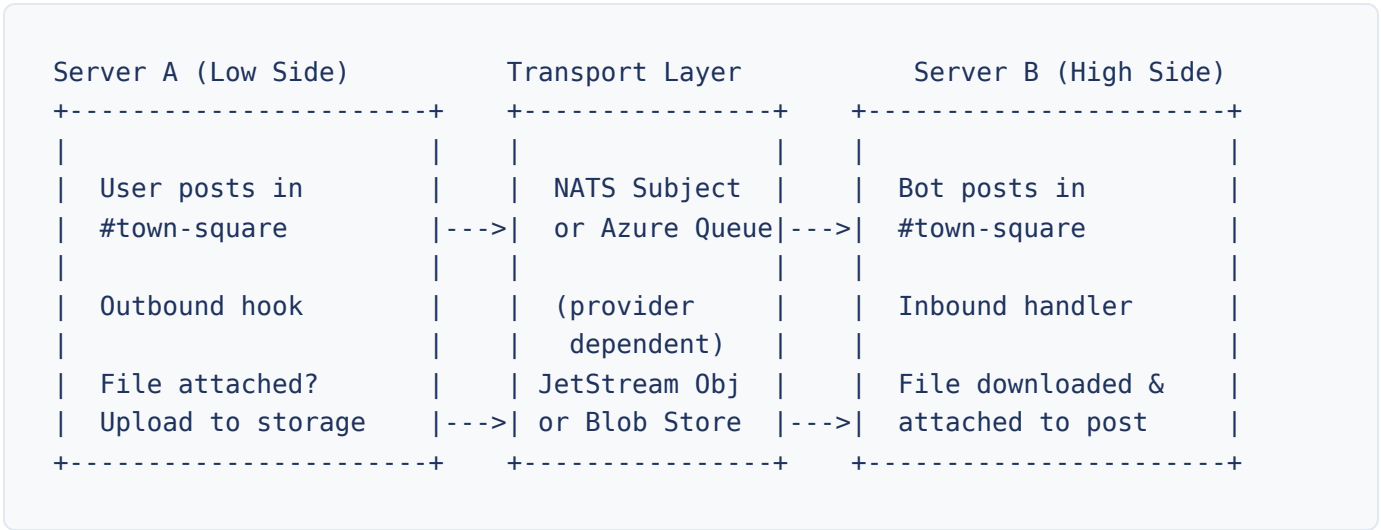
What is Cross Guard?

Cross Guard enables message relay between separate Mattermost servers using NATS or Azure Queue Storage as the transport layer. It is designed for environments where multiple isolated Mattermost instances need to exchange messages across domain boundaries without direct database or API access between servers.

Posts, edits, deletes, reactions, and file attachments flow bidirectionally across server boundaries. Each server maintains its own users, teams, and channels. Cross Guard bridges them at the message level, preserving each server's independence while enabling real-time communication. File attachments are transferred using NATS JetStream Object Store or Azure Blob Storage, depending on the configured provider. File transfer is opt-in per connection.

The plugin integrates with Mattermost's post lifecycle hooks, so relay behavior is transparent to end users. Messages appear in channels as if they were posted locally.

How Federation Works



Two Connection Directions

- **Outbound:** Hooks into Mattermost post lifecycle events (`MessageHasBeenPosted`, `MessageHasBeenUpdated`, `MessageHasBeenDeleted`, reaction events), serializes to a message envelope (JSON or XML, per connection config), and sends via the configured transport.
- **Inbound:** Receives messages from the transport (NATS subscription or Azure queue polling), auto-detects the message format (JSON or XML), deserializes the envelope, and creates, updates, or deletes posts locally.

What Gets Relayed

Event	Relayed?	Notes
New posts	Yes	Including thread replies (root_id preserved)
Post edits	Yes	Updated content replaces original on the remote side
Post deletes	Yes	Matched by original post ID
Reaction adds	Yes	Emoji name and user are relayed. Sync-user reactions are filtered.
Reaction removes	Yes	Matched by emoji name and post ID. Sync-user reactions are filtered.
File attachments	Configurable	Enabled per connection via `file_transfer_enabled`. Supports allow/deny extension filters.
System messages	No	Filtered out to avoid noise
Relay bot posts	No	Loop prevention via `crossguard_relayed` property (posts/edits), sync-user filtering (reactions), and delete flag mechanism (deletes)

Key Concepts

Connections

Named connection configurations (NATS or Azure Queue Storage), inbound or outbound, defined in the System Console. Referenced by name in all commands using the format `direction:name` (e.g., `outbound:relay-to-b`).

Team Initialization

A team must be linked to a connection before any channels in that team can relay messages. This is always the first step. Use `/crossguard init-team` to link a connection.

Channel Initialization

Individual channels within an initialized team are linked to connections. Only linked channels relay messages. Use `/crossguard init-channel` to link a channel.

Inbound Prompts

When an inbound message arrives for a team or channel that hasn't been explicitly accepted, an interactive prompt (Accept/Block buttons) is posted. Team-level prompts appear in town-square; channel-level prompts appear in the target channel. Admins must accept before messages flow. Use `/crossguard reset-prompt` to clear a blocked prompt.

Team Name Rewriting

Maps remote team names to local teams, allowing servers with different team names to relay correctly. Configured per inbound connection using `/crossguard rewrite-team`.

Username Lookup

When enabled (the `UsernameLookup` setting), inbound messages are posted as the matching local user if one exists with the same username. Otherwise, a synthetic relay bot posts them.

Getting Started

1. **Configure connections** in **System Console > Plugins > Cross Guard**. Add connection details for your chosen provider (NATS or Azure Queue Storage). See the Admin Configuration Guide for details.
2. **Initialize a team:** Run `/crossguard init-team [connection-name]` in the team you want to enable.

3. **Initialize channels:** Run `/crossguard init-channel [connection-name]` in each channel that should relay messages.

4. **Verify:** Run `/crossguard status` to confirm the setup and check that connections are active.

Permission Model

Role	Scope	Notes
SYSTEM ADMIN	All commands, all teams/channels. Can configure connections in System Console. Can test connections via API. Sees global status overview.	Always has full access regardless of `RestrictToSystemAdmins` setting.
TEAM ADMIN	`init-team`, `teardown-team`, `init-channel`, `teardown-channel`, `reset-prompt`, `reset-channel-prompt`, `rewrite-team`, `status` within their team.	Access blocked when `RestrictToSystemAdmins` is enabled, unless `AllowTeamAdminRequests` is also enabled (see Request Mode below).
CHANNEL ADMIN	`init-channel`, `teardown-channel` within their channel (team must already be initialized).	Access blocked when `RestrictToSystemAdmins` is enabled, unless `AllowChannelAdminRequests` is also enabled (see Channel Request Mode below).
MEMBER	`status` (team-scoped view), `help`.	Can see which connections are active for their team/channel.

> **Note**
> When `RestrictToSystemAdmins` is enabled in plugin settings, Team Admins and Channel Admins lose access to connection management commands. Only System Admins can manage connections.

> **Request Mode (Team)**

When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins regain limited access: `init-team` submits a link request for System Admin approval (instead of linking directly), and `teardown-team` allows direct unlinking without approval. System Admins receive request DMs with interactive Approve/Deny buttons. See the Admin Configuration page for details.

Channel Request Mode
When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins regain limited access: `init-channel` submits a channel link request for Team Admin approval (instead of linking directly), and `teardown-channel` allows direct unlinking without approval. Team Admins receive request DMs with interactive Approve/Deny buttons. See the Admin Configuration page for details.

Documentation Pages

Slash Command Reference

Complete documentation for all 9 slash commands, including arguments, permissions, and examples.

Admin Configuration Guide

Connection setup for NATS and Azure, relay settings, example configurations, and troubleshooting.

REST API Reference

All 16 REST endpoints with request/response examples and authentication details.

Error Codes

Every numeric `error_code` emitted in Cross Guard logs, grouped by source file, with descriptions and troubleshooting steps.

Transport Interface

Wire protocol reference: message envelope, payload schemas, connection configuration for NATS and Azure Queue Storage, and complete examples.

Threat Model

STRIDE threat analysis, trust zones, data flow documentation, and CDS deployment recommendations.

CDS Whitepaper

Comprehensive architecture, deployment topologies, security analysis, and cross-domain evaluation guide.

Slash Command Reference

All commands for managing Cross Guard relay connections

Overview

- All commands use the trigger `/crossguard`
- Autocomplete is enabled (hints appear as you type)
- Commands return ephemeral responses (only visible to the user who ran them)
- When multiple connections exist and no name is specified, an interactive selection dialog appears

Quick Reference

Command	Description	Permission	Arguments
`init-team [name]`	Link connection to team	TEAM ADMIN+	Optional connection name
`init-channel [name]`	Link connection to channel	CHANNEL ADMIN+	Optional connection name
`teardown-team [name]`	Unlink connection from team	TEAM ADMIN+	Optional connection name
`teardown-channel [name]`	Unlink connection from channel	CHANNEL ADMIN+	Optional connection name
`reset-prompt <name>`	Clear team connection prompt	TEAM ADMIN+	Required connection name
`reset-channel-prompt <name>`	Clear channel connection prompt	TEAM ADMIN+	Required connection name

Command	Description	Permission	Arguments
`rewrite-team [name] [team]`	Set/clear team name rewrite	TEAM ADMIN+	See details
`status`	Show relay status	ANY MEMBER	None
`help`	Show help	ANY MEMBER	None

/crossguard init-team [connection-name]

Link a connection to the current team, enabling cross-domain relay for this team.

Permission	Team Admin or System Admin (blocked by `RestrictToSystemAdmins`; see Request Mode below)
Arguments	`connection-name` (optional). The display key of the connection (e.g., `outbound:relay-to-b`).

Behavior

- If omitted and exactly one connection exists in config, auto-selects it
- If omitted and multiple connections exist, opens an interactive selection dialog with all configured connections
- If specified but not found, returns error listing available connections
- If connection already linked to this team, returns "already linked" message
- On first connection linked to team, adds team to initialized teams list
- Posts announcement in `#town-square`: "Cross Guard connection `outbound:relay-to-b` linked to this team by @username"

> **Request Mode**
> When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins who run this command submit a **link request** instead of linking directly. All System Admins receive a DM with Approve/Deny buttons. The requester is notified of the outcome. If a request is already pending for the same team and connection, the command returns an error.

Examples

```
/crossguard init-team
/crossguard init-team outbound:relay-to-b
```

Error Cases

- `No connections configured. Add connections in the System Console first.`
- `Connection not found: xyz. Available connections: outbound:relay-to-b, inbound:relay-from-a`
- `a request is already pending for this connection` (request mode only)

/crossguard init-channel [connection-name]

Link a connection to the current channel. The team must be initialized first with the same connection.

Permission	Channel Admin, Team Admin, or System Admin (blocked by `RestrictToSystemAdmins`; see Channel Request Mode below)
Arguments	`connection-name` (optional, same auto-select/dialog behavior as init-team)

Behavior

- Validates team is initialized first. If the team does not yet have this specific connection linked, the REST API auto-links the team. The slash command requires explicit team initialization via `/crossguard init-team`.
- If connection not linked at team level, returns: `Connection X is not linked to this team. Run /crossguard init-team first.`
- Adds link emoji prefix to channel header (visible indicator)
- Publishes WebSocket event `channel_connections_updated` to update UI indicators
- Posts announcement in the channel

> **Channel Request Mode**
> When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can run this command to submit a channel connection link request. The request is sent as a DM

to all Team Admins with **Approve** and **Deny** buttons. The channel is not linked until a Team Admin approves the request. Team Admins and System Admins still link channels directly.

Examples

```
/crossguard init-channel
/crossguard init-channel outbound:relay-to-b
```

/crossguard teardown-team [connection-name]

Unlink a connection from the current team.

Permission	Team Admin or System Admin (blocked by `RestrictToSystemAdmins`; see note below)
Arguments	`connection-name` (optional). If omitted and multiple connections linked, opens dialog. If one linked, auto-selects.

Behavior

- If no connections linked: `No connections are linked to this team.`
- If this was the last connection, removes team from initialized teams list
- Posts announcement in `#town-square`
- If a pending connection link request exists for the same team and connection, the request is automatically cancelled and the requester is notified

> **Request Mode**
> When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins can run this command to unlink connections directly (no approval needed for unlinking). This is the only connection management action Team Admins can perform without System Admin approval in request mode.

> **Warning**
> This command does NOT automatically teardown channel connections. Channels that still have this connection linked become orphaned. Run `/crossguard teardown-channel` in affected channels

first.

Example

```
/crossguard teardown-team outbound:relay-to-b
```

/crossguard teardown-channel [connection-name]

Unlink a connection from the current channel.

Permission	Channel Admin, Team Admin, or System Admin (blocked by `RestrictToSystemAdmins`; Channel Admins allowed in channel request mode)
Arguments	`connection-name` (optional, same auto-select/dialog behavior)

Behavior

- If no connections linked: `No connections are linked to this channel.`
- If this was the last connection on the channel, removes link emoji prefix from channel header
- Publishes WebSocket event to update UI indicators
- Posts announcement: "All relays for this channel are now inactive."
- If a pending channel connection request exists for the same channel and connection, the request is automatically cancelled and the requester is notified

> **Channel Request Mode**
> When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can run this command to unlink channel connections directly (no approval needed for unlinking). This is the only channel connection management action Channel Admins can perform without Team Admin approval in channel request mode.

/crossguard reset-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current team. After clearing, a new prompt will appear on the next inbound message.

Permission	Team Admin or System Admin
Arguments	`connection-name` (required)

Behavior

- Looks up prompt in KV store for this team + connection name
- If no prompt found: `No prompt found for connection X on this team.`
- Deletes the prompt entry, allowing the system to create a fresh prompt on the next inbound message

Use Case

An admin accidentally blocked an inbound connection and wants to re-evaluate it.

Example

```
/crossguard reset-prompt relay-from-a
```

/crossguard reset-channel-prompt <connection-name>

Clear a blocked or pending inbound connection prompt for the current channel. Same behavior as `reset-prompt` but operates on channel-level prompts.

Permission	Team Admin or System Admin
Arguments	`connection-name` (required)

Example

```
/crossguard reset-channel-prompt relay-from-a
```

/crossguard rewrite-team [connection-name] [remote-team-name]

Set or clear a remote team name rewrite for an inbound connection. When set, inbound messages with the specified remote team name are routed to this team instead of requiring a matching local team name.

Permission	Team Admin or System Admin
Arguments	`connection-name` (required if multiple inbound connections linked, optional if exactly one). `remote-team-name` (optional; omit to clear the rewrite).

Behavior

- Only works with inbound connections. If no inbound connections linked: `No inbound connections are linked to this team.`
- Sets a rewrite index in the KV store: `connection-name + remote-team-name -> local team ID`
- Returns HTTP 409 (Conflict) if the rewrite index already exists for another team
- Posts audit message in `#town-square` showing who made the change
- When clearing: removes the rewrite index entry and clears `RemoteTeamName` from the team connection metadata

Examples

```
/crossguard rewrite-team relay-from-a remote-team-alpha
/crossguard rewrite-team relay-from-a
```

/crossguard status

Show Cross Guard initialization status for the current team and channel.

Permission	Any team member
Arguments	None

Output Varies by Role

- **Regular users / Team Admins:** Current channel status (relay enabled yes/no), current team status (initialized yes/no), team connections list, channel connections list. Each connection displays file transfer status (e.g., `files off`, `files on`, `files on, allow: .pdf,.docx`).
- **System Admins:** Global overview with all initialized teams (table with display name, team ID, team name, linked connections), all configured connections (table with name, direction, address, auth type, subject, file transfer status), plus current channel status. Warnings shown if connection configuration cannot be parsed.

/crossguard help

Show inline help for all commands. No arguments or special permissions required.

Connection Selection Dialog

When a command needs a connection name and multiple connections are available, an interactive dialog opens automatically.

- Dialog title varies: "Link Connection to Team", "Unlink Connection from Team", etc.
- Dropdown shows all available connections in `direction:name` format
- Dialog submits to `/plugins/crossguard/api/v1/dialog/select-connection`
- State encodes the action and target: `init-team:{teamID}`, `init-channel:{channelID}`, etc.

Admin Configuration Guide

Transport connections, relay settings, and deployment examples

Accessing Plugin Settings

Navigate to **System Console > Plugins > Cross Guard**. The plugin must be enabled before configuration is accessible. Settings are organized into two sections: Connection Configuration and Relay Settings.

Connection Configuration

Understanding Inbound vs Outbound

Direction	Flow	Use When
Outbound	Mattermost → Transport	Your server is the **sender**. Publishes messages to the configured transport.
Inbound	Transport → Mattermost	Your server is the **receiver**. Subscribes to or polls messages from the configured transport.

A server can have both inbound and outbound connections simultaneously for bidirectional relay. Multiple connections of the same direction are supported for multi-server topologies.

Provider Selection

Each connection uses either **NATS** or **Azure Queue Storage** as its transport provider. The provider is selected when creating a connection, and the admin console shows only the fields relevant to the chosen provider.

Provider	Transport	File Transfer	Delivery
NATS	Pub/sub messaging via NATS Core	JetStream Object Store	At-most-once
Azure Queue Storage	Polling-based queue (5s interval)	Azure Blob Storage	At-least-once
Azure Blob Storage	Write-ahead log batched to blobs (periodic flush)	Deferred file blobs alongside batch WAL blobs	At-least-once

Common Connection Fields

Field	JSON Key	Required	Validation	Description
Name	`name`	Yes	Lowercase letters, numbers, and hyphens only. Pattern: `^[a-z0-9]+(-[a-z0-9]+)*$`. Must be unique within its direction (inbound or outbound). The same name may appear in both an inbound and an outbound entry, which is the loopback configuration.	Human-readable identifier. Used in all commands and API calls. Examples: `relay-to-b`, `high-side`, `nats-prod`
Provider	`provider`	Yes	`"nats"`, `"azure-queue"`, or `"azure-`	Transport provider for this connection. Determines which provider-specific fields

Field	JSON Key	Required	Validation	Description
			`blob"`. Empty treated as `"nats"`.	are required. Default: `"nats"`.
File Transfer Enabled	`file_transfer_enabled`	No	Boolean	Enable file attachment relay for this connection. Default: `false`.
File Filter Mode	`file_filter_mode`	No	`""`, `"allow"`, or `"deny"`	Controls whether file extensions are filtered. `allow` = only listed extensions are relayed. `deny` = listed extensions are blocked. Empty = all files pass (default).
File Filter Types	`file_filter_types`	Conditional	Comma-separated extensions. Case-insensitive, leading dot optional.	List of file extensions used by the filter (e.g. `.pdf,.docx,.png`). Required when `file_filter_mode` is `allow` or `deny`.
Message Format	`message_format`	No	`"json"` or `"xml"`. Empty treated as `"json"`.	Wire format for outbound messages. Use `"xml"` when sending through a Cross Domain Solution (CDS) that requires XML. Inbound messages are auto-detected regardless of this setting. Only applies to outbound connections. Default: `"json"`.

NATS Provider Fields

These fields appear when the provider is set to **NATS**. In JSON, they are nested under the `"nats"` key (e.g., `"nats": {"address": "...", "subject": "..."}`).

Field	JSON Key	Required	Validation	Description
Address	`address`	Yes	Non-empty string	NATS server URL. Format: `nats://hostname:port`. Default port is 4222. Examples: `nats://nats.example.com:4222` `nats://10.0.1.50:4222`
Subject	`subject`	Yes	Must start with `crossguard.` prefix	NATS subject to publish/subscribe on. Both sides MUST use the same subject. Examples: `crossguard.relay.messages`, `crossguard.prod.sync`
TLS Enabled	`tls_enabled`	No	Boolean	Enable TLS encryption. Required for production with TLS-enabled NATS servers.
Auth Type	`auth_type`	No	`none`, `token`, or `credentials`. Empty treated as `none`.	Authentication method for the NATS server.
Token	`token`	Conditional	Required when `auth_type` is `token`	Authentication token for token-based auth.
Username	`username`	Conditional	Required when `auth_type` is `credentials`	Username for credential-based auth
Password	`password`	Conditional	Required when `auth_type` is `credentials`	Password for credential-based auth.

Field	JSON Key	Required	Validation	Description
Client Certificate	`client_cert`	No	If provided, `client_key` must also be provided	Path to client TLS certificate file (requires `tls_enabled`).
Client Key	`client_key`	No	If provided, `client_cert` must also be provided	Path to client TLS private key file.
CA Certificate	`ca_cert`	No	-	Path to CA certificate for verifying th NATS server's TLS certificate.

Azure Queue Provider Fields

These fields appear when the provider is set to **Azure Queue Storage**. In JSON, they are nested under the `"azure_queue"` key (e.g., `"azure_queue": {"queue_service_url": "...", "account_name": "...", "account_key": "...", "queue_name": "..."}`).

Field	JSON Key	Required	Validation	Description
Queue Service URL	`queue_service_url`	Yes	Non-empty, valid URL	Azure Queue S Azure, the star `https://<ac` For Azurite (loc `http://azur`
Blob Service URL	`blob_service_url`	Conditional	Required when `file_transfer_enabled` is `true`. Must be a valid URL.	Azure Blob Sto Azure, the star `https://<ac`
Account Name	`account_name`	Yes	Non-empty string	Azure Storage `account_key`
Account Key	`account_key`	Yes	Non-empty string	Azure Storage Treat as a secr under Storage

Field	JSON Key	Required	Validation	Description
Queue Name	`queue_name`	Yes	Non-empty string	Name of the A the queue if it the same queu
Blob Container Name	`blob_container_name`	Conditional	Required when `file_transfer_enabled` is `true`	Name of the A Auto-created if the same conta

Azure Blob Provider Fields

These fields appear when the provider is set to **Azure Blob Storage** (the batched WAL transport, distinct from the Azure Queue provider above). In JSON, they are nested under the `"azure_blob"` key. Instead of a queue, the outbound side batches messages into a write-ahead log blob and flushes it on an interval; the inbound side polls the container, takes a short-lived lease on each batch blob, processes it, and deletes it. File attachments are stored as deferred blobs alongside the batch blobs in the same container.

Field	JSON Key	Required	Validation	Description
Service URL	`service_url`	Yes	Non-empty, valid URL	Azure Blob Storage servic Azure, the standard form i `https://<account>.bl` For Azurite, use `http://azurite:10000`
Account Name	`account_name`	Yes	Non-empty string	Azure Storage account na `account_key` to form a s
Account Key	`account_key`	Yes	Non-empty string	Azure Storage account sh (base64). Treat as a secre
Blob Container Name	`blob_container_name`	Yes	Non-empty string	Name of the Azure Blob c batch WAL blobs and (whe deferred file blobs. Auto-cr Both sides must use the s

Field	JSON Key	Required	Validation	Description
Flush Interval	`flush_interval_seconds`	No	Integer. If set, must be at least `5`.	How often the outbound si write-ahead log blob and u seconds. Lower values rec at the cost of more blob op
Blob Lock Max Age	`blob_lock_max_age_seconds`	No	Integer. If set, must be at least `30`.	Maximum age of a blob lea side considers it stale and Default: `300` seconds (5 crashed inbound node fror

Azure Service Bus Provider Fields

These fields appear when the provider is set to **Azure Service Bus**. In JSON, they are nested under the `"azure_servicebus"` key. Service Bus uses an AMQP-based queue for message relay with PeekLock semantics: the plugin completes messages on successful handler invocation and abandons them on handler error (Service Bus then increments delivery count and routes to the dead-letter queue after the queue-configured `MaxDeliveryCount`).

Important: The plugin does *not* auto-create the queue. Pre-create it in the Azure Portal or via ARM/Terraform with your chosen `LockDuration` (default 60 s, maximum 5 min) and `MaxDeliveryCount` before saving the connection. Lock duration is an entity property; the plugin cannot configure it.

Why the blob fields are named differently from the Azure Queue provider: Service Bus and Azure Storage are separate Azure resources with independent credentials. The Azure Queue provider shares one storage account for both queues and blobs; the Service Bus provider needs a distinct credential pair for its Blob sidecar (which is why you see `blob_account_name` / `blob_account_key` here instead of the unqualified `account_name` / `account_key`).

Field	JSON Key	Required	Validation	De
Connection String	`connection_string`	Yes	Non-empty string	SA `E` `<r` re

Field	JSON Key	Required	Validation	De
Queue Name	`queue_name`	Yes	Up to 260 characters; letters, digits, periods, hyphens, underscores, and forward slashes.	Na `t`
Blob Service URL	`blob_service_url`	Only when `file_transfer_enabled` is true	Valid URL	Az
Blob Account Name	`blob_account_name`	Only when `file_transfer_enabled` is true	Non-empty string	St
Blob Account Key	`blob_account_key`	Only when `file_transfer_enabled` is true	Non-empty string	Sh
Blob Container Name	`blob_container_name`	Only when `file_transfer_enabled` is true	Non-empty string	Bl
Max Message Size	`max_message_size_bytes`	No	Integer, 1024 to 104857600. Default `192000`.	Ma bo
Blob Poll Interval	`blob_poll_interval_seconds`	No	Integer, ≥ 1. Default `15`.	In

Dead-Letter Queue (DLQ) behaviour: If handler errors cause redelivery count to exceed the queue's `MaxDeliveryCount`, Azure Service Bus auto-moves the message to the queue's dead-letter sub-queue. The plugin does NOT inspect or process DLQ messages; they stop appearing in receive polls. Use the Azure Portal, `az servicebus queue message` CLI, or Azure monitoring

alerts to inspect the DLQ. The plugin logs a WARN with error code `26004` (`ServiceBusRedelivery`) on every redelivery so operators can detect poison-message buildup before DLQ moves happen.

Azure Service Principal Authentication

All three Azure providers (Queue, Blob, Service Bus) support **Azure AD Service Principal authentication** as an alternative to shared keys / SAS connection strings. Service Principal uses the AAD `client_credentials` grant: an application registered in Azure AD authenticates with a tenant ID, a client (application) ID, and a client secret. The plugin acquires bearer tokens from AAD and uses them to call the Azure data plane.

Why this matters: Shared keys grant full account access and cannot be scoped or rotated centrally. Service Principal + Azure RBAC lets you grant a single AAD identity exactly the permissions it needs on a specific queue, container, or namespace, and rotation happens in AAD without touching plugin config.

Phase 1 Deployment Guidance

This release supports the **Client Secret** flow only. Client certificate auth and Managed Identity are tracked as later phases. From a compliance / threat-model perspective:

- **Acceptable for:** development, staging, and production environments paired with a strict secret-rotation policy (90-day maximum). Federal customers using Azure Government with rotation tooling can deploy Phase 1 in production.
- **Not the recommended posture for:** high-side ATO environments where shared-secret material at rest is a finding. For those environments, wait for Phase 2 (certificate auth via Azure Key Vault + CSI driver) or use the Mattermost env-var substitution mechanism described below to keep the secret out of plugin config entirely.

Known Exposure: Secret in Plugin Config (Phase 1B Follow-Up)

The Mattermost `GET /api/v4/config` endpoint returns plugin custom-settings as cleartext to any System Admin. This affects **all** Crossguard secret fields (`account_key`, `connection_string`, `blob_account_key`, `client_secret`): an admin opening the System Console will see the secret value in their browser's network tab and any `mmctl config show` output will include it. A structural fix that replaces the cleartext with a reference is tracked as Phase 1B.

Immediate Federal mitigation: inject the entire connections JSON via Mattermost's plugin-settings env-var substitution. Setting

`MM_PLUGINSETTINGS_PLUGINS_CROSSGUARD_OUTBOUNDCONNECTIONS` (and the inbound counterpart) overrides whatever is stored in plugin config at load time; the value can come from a Kubernetes Secret, a CSI volume + init container, systemd `LoadCredential=`, or HashiCorp Vault. The credential then lives only in the Mattermost process environment, not in `config.json` on disk, not in the configuration database, and not in `GET /api/v4/config` responses. This is the same mechanism Mattermost uses for every other plugin secret.

Common Service Principal Fields

The following fields are shared across all three Azure providers when `auth_mode` is `service-principal`:

Field	Required	Type	Description
`auth_mode`	No	String enum. Queue/Blob: `"shared-key"` (default) or `"service-principal"`. Service Bus: `"connection-string"` (default) or `"service-principal"`.	Selects the credential type. Empty defaults backward compatibility (existing configs ke
`azure_cloud`	No	String enum: `"public"` (default), `"usgov"`, `"china"`.	Selects the Azure cloud environment. Pass for credential acquisition. For Queue and B plane SDK client options. For Service Bus from the FQDN namespace, so only the cr option.
`tenant_id`	Yes (SP)	String. GUID or domain form (e.g., `contoso.onmicrosoft.com`).	Azure AD directory (tenant) ID.
`client_id`	Yes (SP)	String (GUID).	Azure AD application (client) ID for the Ser
`client_secret`	Yes (SP)	String (secret).	Inline client secret value. Required in SP n of plugin config on disk, inject the whole co `MM_PLUGINSETTINGS_PLUGINS_CROSSGU` / `INBOUNDCONNECTIONS` env var (see Ph

Azure Service Bus – Additional SP Field

Field	Required	Type	Description
`service_bus_namespace`	Yes (SP)	String (FQDN).	Fully-qualified Service Bus namespace, e.g., `myns.servicebus.windows.net` for Public, `myns.servicebus.usgovcloudapi.net` for US Government. The SAS connection string used to encode this for the legacy mode; in SP mode it must be supplied separately. If you accidentally include the `sb://` scheme, it is stripped.

Minimum Required Azure RBAC Roles

Grant the Service Principal these data-plane roles at the resource scope. The plugin **skips auto-creation** of queues/containers in SP mode — you must pre-provision them via the Azure Portal, ARM template, or `az` CLI. This means the operator does not need management-plane (Owner / Contributor) roles, which is the correct least-privilege posture.

- **Azure Queue provider:** `Storage Queue Data Contributor` on the Queue Storage account. If file transfer is enabled, also grant `Storage Blob Data Contributor` on the same account.
- **Azure Blob provider:** `Storage Blob Data Contributor` on the Blob Storage account.
- **Azure Service Bus provider:** `Azure Service Bus Data Sender` AND `Azure Service Bus Data Receiver` on the queue (or on the namespace). If file transfer is enabled and the blob sidecar inherits the parent SP credential (the default in SP mode), the same SP also needs `Storage Blob Data Contributor` on the blob storage account.

Owner-tier roles are NOT required. If you see startup errors about `AuthorizationPermissionMismatch` on a Create call, double-check that the queue/container has been pre-provisioned and that the SP has the correct data-plane role.

Service Bus Blob Sidecar Credential Inheritance (SP Mode)

When Azure Service Bus is in SP mode the blob sidecar **inherits the parent SP credential**: do NOT set `blob_account_name` or `blob_account_key`, validation rejects the save if you do. This empty-fields rule applies unconditionally in SP mode, even when `file_transfer_enabled` is false, so a stored shared-key cannot linger across a file-transfer toggle. When file transfer is enabled, provide `blob_service_url` and `blob_container_name`. The same Service Principal must have the appropriate roles on both the Service Bus namespace and the blob storage account. Mixed-credential deployments (one SP for AMQP, a different SP or shared key for blob) are deferred to Phase 2.

Azure Government (Sovereign Cloud) Setup

To configure a connection for Azure US Government:

1. Set `azure_cloud` to `usgov`.
2. Use the US Gov endpoints in `queue_service_url`, `blob_service_url`, `service_url`, or `service_bus_namespace`: `*.queue.core.usgovcloudapi.net`, `*.blob.core.usgovcloudapi.net`, `*.servicebus.usgovcloudapi.net`.
3. Register the Service Principal in the Azure US Government AAD tenant (not the commercial tenant). Grant the data-plane RBAC roles described above.

The plugin passes the cloud configuration to `azidentity` so token acquisition routes to `login.microsoftonline.us` automatically; you do not need to override an authority host.

Save-Time Credential Probe

When you save a plugin configuration that includes one or more SP connections, the plugin acquires a token from AAD for each SP connection before the save commits. Bad tenant/client/secret values fail the save with a sanitized error message instead of producing repeated runtime failures during the relay loop. The probe has a 10-second timeout. Errors are scrubbed of any credential material before being logged or surfaced to the admin UI.

Secret Hydration in the Admin Console

When you open an existing connection in the System Console, password fields (including `client_secret`, `account_key`, `connection_string`, `blob_account_key`) are hydrated with a distinctive sentinel token (`__CROSSGUARD_SECRET_UNCHANGED__`) instead of the stored cleartext. The token is rendered as bullets in the password input and never round-trips to a browser as a real secret. If you save the form without editing a password field, the sentinel is sent back to the server and the stored value is preserved.

- **To rotate a secret:** paste the new value into the field and save.
- **To leave a secret unchanged:** do nothing — do not delete the field contents.
- **To clear a stored secret** (for example, when migrating from shared-key to service-principal): clear the field and save. An empty inbound value is honored as an explicit clear.

Validation Rules Summary

1. Names must be unique within each direction (inbound or outbound). The same bare name may appear in both lists, which is the loopback configuration where one server is both

publishing and subscribing on the same transport

2. Names must be lowercase alphanumeric with hyphens (no spaces, underscores, uppercase)
3. Subject must always start with `crossguard.`
4. Both sides of a relay must use the same NATS subject
5. Auth credentials must be complete (both username AND password for credentials auth)
6. TLS cert and key must be provided as a pair (enforced at config save time)
7. Azure Queue connections require `queue_service_url` and `queue_name`; shared-key mode additionally requires `account_name` and `account_key`
8. Azure Queue connections require `blob_service_url` and `blob_container_name` when file transfer is enabled
9. Azure Blob connections require `service_url` and `blob_container_name`; shared-key mode additionally requires `account_name` and `account_key`
10. Azure Blob `flush_interval_seconds` must be at least 5 when set; `blob_lock_max_age_seconds` must be at least 30 when set
11. Azure Service Bus connections require `queue_name`; connection-string mode additionally requires `connection_string`; service-principal mode additionally requires `service_bus_namespace`
12. Azure Service Bus `queue_name` is capped at 260 characters and allows only `A-Z a-z 0-9 . _ - /`
13. Azure Service Bus connections with file transfer enabled also require `blob_service_url` and `blob_container_name`; in connection-string mode also `blob_account_name` and `blob_account_key`. In service-principal mode `blob_account_name` and `blob_account_key` must be **empty** (sidecar inherits parent SP); this empty-fields rule applies unconditionally in SP mode, including when file transfer is disabled, so stale shared-key material cannot survive a file-transfer toggle
14. When `auth_mode` is `service-principal`: `tenant_id` (GUID or FQDN), `client_id`, and `client_secret` are required; the corresponding legacy field (`account_key` / `connection_string`) must be empty
15. When `auth_mode` is set, `azure_cloud` must be one of `public` , `usgov` , or `china` when present; empty defaults to `public`

Relay Settings

Username Lookup

Setting	`UsernameLookup`
Type	Boolean
Default	`true`

When enabled: Inbound messages look for a local user with the same username as the remote sender. If found, the message is posted as that local user. This creates a seamless experience where messages appear to come from the correct person.

When disabled: All inbound messages are posted by the Cross Guard relay bot, with the remote username shown in the message formatting. Use this if usernames don't match across servers.

Restrict to System Admins

Setting	`RestrictToSystemAdmins`
Type	Boolean
Default	`false`

When enabled: Only System Admins can run `init-team`, `teardown-team`, `init-channel`, `teardown-channel`, `reset-prompt`, `reset-channel-prompt`, and `rewrite-team`. Team Admins and Channel Admins are denied access.

When disabled: Team Admins can manage team connections, Channel Admins can manage channel connections (the default permission hierarchy).

> **Tip**
> Enable this setting in high-security environments where only centralized admins should control cross-domain relay. Pair it with `AllowTeamAdminRequests` below to let Team Admins request connections without granting them direct control.

Allow Team Admin Requests

Setting	`AllowTeamAdminRequests`

Type	Boolean
Default	`false`
Requires	`RestrictToSystemAdmins` must also be enabled

When enabled (and `RestrictToSystemAdmins` is also enabled): Team Admins can submit connection link requests via `/crossguard init-team` or the REST API. The request is sent as a direct message to every System Admin with interactive **Approve** and **Deny** buttons. Team Admins can also unlink connections directly with `/crossguard teardown-team` (no approval needed for unlinking).

When disabled: Team Admins are fully blocked from connection management when `RestrictToSystemAdmins` is enabled. This has no effect when `RestrictToSystemAdmins` is disabled, because Team Admins already have direct access.

> **Request workflow**
> When a Team Admin submits a link request, all System Admins receive a DM from the Cross Guard bot with the requester's name, team, and connection. The DM contains **Approve** and **Deny** buttons. Approving executes the link immediately. Denying opens an optional reason dialog. The requester is notified of the outcome via DM. If the connection is unlinked while a request is pending, the request is automatically cancelled.

Allow Channel Admin Requests

Setting	`AllowChannelAdminRequests`
Type	Boolean
Default	`false`
Requires	`RestrictToSystemAdmins` must also be enabled

When enabled (and `RestrictToSystemAdmins` is also enabled): Channel Admins can submit channel connection link requests via `/crossguard init-channel` or the REST API. The request is sent as a direct message to every Team Admin with interactive **Approve** and **Deny** buttons. Channel Admins can also unlink channel connections directly with `/crossguard teardown-channel` (no approval needed for unlinking).

When disabled: Channel Admins are fully blocked from channel connection management when `RestrictToSystemAdmins` is enabled. This has no effect when `RestrictToSystemAdmins` is disabled, because Channel Admins already have direct access.

> **Request workflow**
> When a Channel Admin submits a channel link request, all Team Admins for that team receive a DM from the Cross Guard bot with the requester's name, team, channel, and connection. The DM contains **Approve** and **Deny** buttons. Approving executes the channel link immediately. Denying opens an optional reason dialog. The requester is notified of the outcome via DM. If the connection is unlinked while a request is pending, the request is automatically cancelled.

File Transfer Settings

File transfer is configured per connection, not globally. Each connection can independently enable or disable file attachment relay and apply extension filters.

Requirements

NATS Provider

- **JetStream-enabled NATS server:** File transfer uses NATS JetStream Object Store. Core NATS alone is not sufficient. The NATS server must have JetStream enabled.
- **Object Store bucket:** The plugin auto-creates a `crossguard-files` bucket with a 1-hour TTL on first use. No manual NATS configuration is needed.
- **Max file size:** Governed by the Mattermost server's `FileSettings.MaxFileSize` configuration (default 100 MB). Files exceeding this limit are silently skipped with a log warning.

Azure Provider

- **Blob container:** File transfer uses Azure Blob Storage. The plugin auto-creates the container specified in `blob_container_name` if it does not exist.
- **Blob metadata:** Headers (post ID, connection name, filename) are stored as Base64-encoded JSON in the blob's `crossguard_headers` metadata key.
- **Polling interval:** The inbound blob watcher polls every 15 seconds. Files are deleted after successful processing.

- **Max file size:** Same as NATS: governed by the Mattermost server's `FileSettings.MaxFileSize` configuration.

Filter Examples

Goal	file_filter_mode	file_filter_types
Allow all files (default)	`""`	`""`
Allow only PDFs and images	`"allow"`	`".pdf,.png,.jpg,.gif"`
Block executables	`"deny"`	`".exe,.bat,.sh,.cmd"`

> **Note**
> For bidirectional file relay, both the outbound and inbound connections must have `file_transfer_enabled` set to `true`. File filters are evaluated independently on each side, so the outbound connection can filter what it sends and the inbound connection can filter what it accepts.

Testing Connections

The admin console UI includes a "Test Connection" button for each configured connection.

- **API endpoint:** `POST /plugins/crossguard/api/v1/test-connection` (System Admin only)
- **NATS outbound test:** Publishes a message with a unique ID and confirms flush
- **NATS inbound test:** Subscribes to the subject and confirms the subscription is active
- **Azure test:** Validates the connection string by enqueuing and deleting a test message
- Tests use a temporary connection (does not affect the running plugin's connections)
- The `direction` query parameter is NATS-specific; Azure tests validate connectivity regardless of direction

Example Configurations

Dual-Server Setup (Server A sends to Server B)

Server A (sender)

```
Outbound Connections:
[{
  "name": "relay-to-b",
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
    "subject": "crossguard.relay.messages",
    "auth_type": "token",
    "token": "shared-secret-token"
  },
  "file_transfer_enabled": true,
  "file_filter_mode": "allow",
  "file_filter_types": ".pdf,.docx,.png"
}]

Inbound Connections: []
```

Server B (receiver)

```
Inbound Connections:
[{
  "name": "relay-from-a",
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
    "subject": "crossguard.relay.messages",
    "auth_type": "token",
    "token": "shared-secret-token"
  },
  "file_transfer_enabled": true
}]

Outbound Connections: []
```

Bidirectional Setup (Both servers send and receive)

Server A

```
Outbound: [{"name": "to-b", "provider": "nats", "nats": {"address": "nats://nats:4222"
Inbound:  [{"name": "from-b", "provider": "nats", "nats": {"address": "nats://nats:422
```

Server B

```
Outbound: [{"name": "to-a", "provider": "nats", "nats": {"address": "nats://nats:4222"
Inbound:  [{"name": "from-a", "provider": "nats", "nats": {"address": "nats://nats:422
```

> **Note**
> Each direction uses a different subject to avoid echo loops. Server A publishes on `crossguard.a-to-b` and Server B subscribes to it, and vice versa.

Docker Dev Environment

- **Server A:** `http://low.test:8075` (admin/password, usera/password, Team: Test A)
- **Server B:** `http://high.test:8076` (admin/password, userb/password, Team: Test B)
- **NATS:** `nats://nats:4222` (from plugin's perspective inside Docker network)
- **Setup:** `make docker-setup`, **Deploy:** `make deploy`

Azure Queue Storage Setup

Server A (sender via Azure Queue)

```
Outbound Connections:
[{
  "name": "azure-to-b",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  },
```

```
  "file_transfer_enabled": true
}]
```

Server B (receiver via Azure Queue)

```
Inbound Connections:
[{
  "name": "azure-from-a",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "blob_service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay",
    "blob_container_name": "crossguard-files"
  },
  "file_transfer_enabled": true
}]
```

Azure Blob variant (batched WAL transport)

```
Outbound Connections:
[{
  "name": "azure-blob-to-b",
  "provider": "azure-blob",
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal",
    "flush_interval_seconds": 60,
    "blob_lock_max_age_seconds": 300
  },
  "file_transfer_enabled": true
}]
```

> **Note**
> Both sides share the same Azure Storage account, queue name, and blob container. The outbound side enqueues messages and uploads blobs; the inbound side polls and processes them.

Troubleshooting

Problem	Solution
"No connections configured"	Add connections in System Console > Plugins > Cross Guard first.
NATS connection test fails	Verify NATS server is reachable, check address/port, verify auth credentials match.
Messages not relaying	Run `/crossguard status`. Verify both team AND channel are initialized. Check that transport identifiers match on both sides (NATS subject for NATS connections, queue name for Azure connections).
"Connection not found"	Connection name is case-sensitive and uses `direction:name` format (e.g., `outbound:relay-to-b`).
Orphaned connections	If a connection name is changed in config, existing team/channel links reference the old name. Teardown and re-init with the new name.
Inbound messages blocked	Check if an inbound prompt was blocked. Run `/crossguard reset-prompt <name>` to clear it and re-evaluate.
Wrong team receives messages	Check team name rewrite settings with `/crossguard status`. Remote team names must match or be rewritten.
Files not transferring	Verify `file_transfer_enabled` is `true` on both outbound and inbound connections. For NATS, ensure the server has JetStream enabled (not just core NATS). For Azure, verify `blob_container_name` is set on both outbound and inbound connections.
"File semaphore full" log warnings	Too many concurrent file transfers. The limit is 32 simultaneous file operations. Large files or slow connections can cause congestion. Consider reducing file sizes or upgrading network bandwidth.
File blocked by filter	Check `file_filter_mode` and `file_filter_types` on the connection. An `allow` filter rejects extensions not in the list; a `deny` filter rejects extensions in the list.

Problem	Solution
File appears after text message	Normal behavior. Files arrive asynchronously via the file watcher (NATS Object Store watcher or Azure Blob poller). The inbound watcher retries post mapping lookup up to 3 times with backoff to wait for the text message to be processed first.
Large files silently dropped	Files larger than the server's `MaxFileSize` setting (default 100 MB) are skipped with a log warning. Check `FileSettings.MaxFileSize` in the Mattermost server configuration.
Azure connection test fails	Verify the connection string is correct. Check that the storage account is accessible from the server. Ensure the account key has not been rotated.
Azure messages delayed	Azure Queue Storage uses polling (5-second intervals for messages, 15-second intervals for files). This is normal. For lower latency, use the NATS provider.
Azure message too large	Azure Queue Storage has a 48,000-byte message size limit (before Base64 encoding to the 64 KB wire limit). Messages exceeding this limit have their text truncated to fit. Long posts may lose content.
Duplicate messages with Azure	Azure Queue provides at-least-once delivery. The plugin uses idempotency checks (post ID mapping) to prevent duplicates, but brief visibility timeout races can cause rare duplicates during error recovery.
Azure blob files not appearing	Verify `blob_container_name` is set on both sides. Check that `file_transfer_enabled` is `true` on both outbound and inbound connections. The blob watcher polls every 15 seconds.

REST API Reference

All Cross Guard REST endpoints with request and response examples

Overview

Base Path	`/plugins/crossguard/api/v1/`
Authentication	All endpoints require the `Mattermost-User-Id` header (set automatically by Mattermost for authenticated requests).
Request Limit	5 MB max request body
Response Format	JSON
Error Format	`{"error": "message"}` with appropriate HTTP status code

Connection Testing

POST `/api/v1/test-connection` SYSTEM ADMIN

Test a connection configuration before saving. Supports NATS, Azure Queue, Azure Blob, and Azure Service Bus providers.

Query Parameters

`direction`	Optional. `inbound` or `outbound` (defaults to `outbound`). NATS-specific; ignored for Azure Queue, Azure Blob, and Azure Service Bus connections.

Request Body (NATS)

```
{
  "name": "relay-to-b",
  "provider": "nats",
  "nats": {
    "address": "nats://nats.example.com:4222",
```

```
    "subject": "crossguard.relay.messages",
    "tls_enabled": false,
    "auth_type": "token",
    "token": "my-secret-token"
  }
}
```

Request Body (Azure Queue)

```
{
  "name": "azure-relay",
  "provider": "azure-queue",
  "azure_queue": {
    "queue_service_url": "https://mystorageaccount.queue.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "queue_name": "crossguard-relay"
  }
}
```

Request Body (Azure Blob)

```
{
  "name": "azure-blob-relay",
  "provider": "azure-blob",
  "azure_blob": {
    "service_url": "https://mystorageaccount.blob.core.windows.net",
    "account_name": "mystorageaccount",
    "account_key": "base64-account-key",
    "blob_container_name": "crossguard-wal"
  }
}
```

Request Body (Azure Service Bus)

```
{
  "name": "azure-servicebus-relay",
  "provider": "azure-servicebus",
  "azure_servicebus": {
    "connection_string": "Endpoint=sb://myns.servicebus.windows.net/;SharedAcces
    "queue_name": "crossguard-relay"
```

```
  }
}
```

Validation

NATS

- `nats.address` required
- `nats.subject` required and must start with `crossguard.`
- `nats.auth_type` must be `none`, `token`, `credentials`, or empty
- `nats.token` required if auth_type is `token`
- `nats.username` + `nats.password` required if auth_type is `credentials`

Azure Queue

- `azure_queue.queue_service_url` required
- `azure_queue.queue_name` required
- `azure_queue.account_name` and `azure_queue.account_key` required when `azure_queue.auth_mode` is empty or `shared-key`; **must be empty** when `auth_mode` is `service-principal`
- When `file_transfer_enabled` is true, `blob_service_url` and `blob_container_name` are also required

Azure Blob

- `azure_blob.service_url` required
- `azure_blob.blob_container_name` required
- `azure_blob.account_name` and `azure_blob.account_key` required when `azure_blob.auth_mode` is empty or `shared-key`; **must be empty** when `auth_mode` is `service-principal`

Azure Service Bus

- `azure_servicebus.queue_name` required; at most 260 characters; allowed characters: letters, digits, periods, hyphens, underscores, and forward slashes
- `azure_servicebus.connection_string` required when `auth_mode` is empty or `connection-string`; **must be empty** when `auth_mode` is `service-principal`
- `azure_servicebus.service_bus_namespace` required when `auth_mode` is `service-principal` (FQDN-shaped, e.g., `myns.servicebus.windows.net`; the `sb://` scheme is stripped if present)

- When `file_transfer_enabled` is true, `blob_service_url` and `blob_container_name` are required. In `connection-string` mode, `blob_account_name` and `blob_account_key` are also required; in `service-principal` mode they **must be empty** (the blob sidecar inherits the parent SP credential)
- `azure_servicebus.max_message_size_bytes` (optional) must be between 1024 and 104857600 when set
- `azure_servicebus.blob_poll_interval_seconds` (optional) must be at least 1 when set

Service Principal Auth (all three Azure providers)

When the per-provider `auth_mode` is `service-principal`, the request body must additionally include the shared SP fields (`tenant_id` , `client_id` , `client_secret` , and optionally `azure_cloud`). See the Azure Service Principal Authentication section in the admin guide for the full field reference and minimum RBAC roles. A 400 response is returned when SP validation fails (bad tenant format, missing client_id, missing client_secret, or cross-mode field leakage).

Success Response (outbound) - 200

```
{"status": "ok", "message": "Test message sent with ID: abc123", "id": "abc123"}
```

Success Response (inbound) - 200

```
{"status": "ok", "message": "Connected and subscribed to subject successfully"}
```

Success Response (Azure Queue) - 200

```
{"status": "ok", "message": "Azure Queue Storage connection test successful"}
```

Success Response (Azure Blob) - 200

```
{"status": "ok", "message": "Azure Blob Storage connection test successful"}
```

Success Response (Azure Service Bus) - 200

```
{"status": "ok", "message": "Azure Service Bus connection test successful"}
```

Error Responses

Status	Meaning
`400`	Validation error
`401`	Not authenticated
`403`	Not system admin
`502`	Transport connection failed (NATS unreachable, or Azure credentials/service URL invalid)

Team Management

POST `/api/v1/teams/{team_id}/init` TEAM ADMIN+

Link a connection to a team (API equivalent of `/crossguard init-team`).

Path Parameters

`team_id`	26-character Mattermost team ID

Query Parameters

`connection_name`	Optional. Connection in `direction:name` format (e.g., `outbound:relay-to-b`).

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "team_name": "test-a",
  "connection_name": "outbound:relay-to-b"
}
```

Request Mode Response - 200

When the caller is a Team Admin and both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, the endpoint submits a link request instead of linking directly:

```
{
  "status": "request_submitted",
  "message": "Your request to link connection `outbound:relay-to-b` ...",
  "team_id": "abc123...",
  "connection_name": "outbound:relay-to-b"
}
```

Error Responses

Status	Meaning
`400`	Invalid team_id, no connections configured, or connection not found (includes `connections` array of available names)
`403`	Insufficient permissions
`404`	Team not found
`409`	A request is already pending for this connection (request mode only)
`500`	Internal error

POST `/api/v1/teams/{team_id}/teardown` TEAM ADMIN+

Unlink a connection from a team. If a pending connection link request exists for the same team and connection, it is automatically cancelled.

Path Parameters

`team_id`	26-character Mattermost team ID

Query Parameters

`connection_name`	Optional. Connection to unlink.

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "team_name": "test-a",
  "connection_name": "outbound:relay-to-b"
}
```

POST /api/v1/channels/{channel_id}/init CHANNEL ADMIN+

Link a connection to a channel (team auto-inits if needed).

Path Parameters

`channel_id`	26-character Mattermost channel ID

Query Parameters

`connection_name`	Optional. Connection in `direction:name` format.

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "connection_name": "outbound:relay-to-b"
}
```

Channel Request Mode Response - 200

When `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are both enabled and the caller is a Channel Admin (not a Team Admin or System Admin), the endpoint submits a channel connection request instead of linking directly:


```
{
  "status": "request_submitted",
  "message": "Your request to link connection ... has been submitted for team adm
  "channel_id": "xyz789...",
  "connection_name": "outbound:relay-to-b"
}
```


Error Responses

Status	Meaning
`400`	Invalid input or no connections
`403`	Insufficient permissions
`404`	Channel not found
`409`	A channel request is already pending for this connection
`500`	Internal error

POST `/api/v1/channels/{channel_id}/teardown` CHANNEL ADMIN+

Unlink a connection from a channel. If a pending channel connection request exists for the same connection, it is automatically cancelled.

Path Parameters

`channel_id`	26-character Mattermost channel ID

Query Parameters

`connection_name`	Optional. Connection to unlink.

Success Response - 200

```
{
  "status": "ok",
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "connection_name": "outbound:relay-to-b"
}
```

Status

GET `/api/v1/status` SYSTEM ADMIN

Global status of all initialized teams and transport connections.

Response - 200

```
{
  "teams": [
    {
      "team_id": "abc123...",
      "team_name": "test-a",
      "display_name": "Test A",
      "linked_connections": [
        {"direction": "outbound", "connection": "relay-to-b"}
      ]
    }
  ],
  "connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "provider": "nats",
      "address": "nats://nats:4222",
      "auth_type": "token",
```

```
      "subject": "crossguard.relay.messages",
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    },
    {
      "name": "azure-relay",
      "direction": "outbound",
      "provider": "azure-queue",
      "queue_name": "crossguard-relay",
      "file_transfer_enabled": false,
      "message_format": "json"
    },
    {
      "name": "azure-blob-relay",
      "direction": "outbound",
      "provider": "azure-blob",
      "blob_container_name": "crossguard-wal",
      "file_transfer_enabled": false,
      "message_format": "json"
    },
    {
      "name": "azure-servicebus-relay",
      "direction": "outbound",
      "provider": "azure-servicebus",
      "queue_name": "crossguard-relay",
      "file_transfer_enabled": false,
      "message_format": "json"
    }
  ]
}
```


> **Note**
> Connection details are redacted. Tokens, passwords, and TLS cert paths are not included.

GET `/api/v1/teams/{team_id}/status` TEAM MEMBER

Initialization and connection status for a specific team.

Response - 200

```
{
  "team_id": "abc123...",
  "team_name": "test-a",
  "team_display_name": "Test A",
  "initialized": true,
  "request_mode": false,
  "linked_connections": [
    {"direction": "outbound", "connection": "relay-to-b"}
  ],
  "connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "request_pending": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```

Response Fields

`orphaned`	`true` when the connection is linked to the team but no longer exists in the plugin configuration.
`remote_team_name`	Set when a team name rewrite is configured for this inbound connection.
`request_mode`	`true` when the calling user is a Team Admin and the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled). Omitted for System Admins.
`request_pending`	`true` when a link request is pending for this unlinked connection. Only populated when the plugin is in request mode.

GET `/api/v1/channels/{channel_id}/status` CHANNEL MEMBER

Connection status for a specific channel.

Response - 200

```
{
  "channel_id": "xyz789...",
  "channel_name": "town-square",
  "channel_display_name": "Town Square",
  "team_name": "Test A",
  "channel_request_mode": true,
  "team_connections": [
    {
      "name": "relay-to-b",
      "direction": "outbound",
      "linked": true,
      "orphaned": false,
      "request_pending": false,
      "file_transfer_enabled": true,
      "file_filter_mode": "allow",
      "file_filter_types": ".pdf,.docx",
      "message_format": "json"
    }
  ]
}
```

Channel request mode fields

`channel_request_mode` (top-level) is `true` when the caller is a Channel Admin operating in channel request mode (not a Team Admin or System Admin). `request_pending` (per connection) is `true` when an unresolved channel link request exists for that connection. Both fields are omitted when not applicable.

Note

Returns an error for DM and group message channels: "Cross Guard is not available for direct or group messages"

GET /api/v1/channels/connections AUTHENTICATED

Bulk query which channels have active Cross Guard connections.

Query Parameters

`ids`	Required. Comma-separated channel IDs (max 2048 IDs).

Response - 200

```
{
  "channel_id_1": "outbound:relay-to-b,inbound:relay-from-a",
  "channel_id_2": "outbound:relay-to-b"
}
```

- Only returns channels the user is a member of
- Invalid IDs and channels without connections are omitted
- Used internally by the webapp to show channel indicators

Team Name Rewriting

POST /api/v1/teams/{team_id}/rewrite TEAM ADMIN+

Set a remote team name rewrite for an inbound connection.

Request Body

```
{
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Validation

- Both `connection` and `remote_team_name` must be non-empty
- Connection must be inbound and linked to this team

Success Response - 200

```
{
  "status": "ok",
  "team_id": "abc123...",
  "connection": "relay-from-a",
  "remote_team_name": "remote-team-alpha"
}
```

Error Responses

Status	Meaning
`400`	Validation error, connection not inbound, or not linked
`403`	Insufficient permissions
`409`	Rewrite index already exists for this connection + remote_team_name pair on another team
`500`	Internal error

DELETE /api/v1/teams/{team_id}/rewrite TEAM ADMIN+

Clear a remote team name rewrite.

Query Parameters

`connection`	Required. Connection name to clear the rewrite for.

Success Response - 200

```
{
  "status": "ok",
```

```
  "team_id": "abc123...",
  "connection": "relay-from-a"
}
```

Connection Prompts

> **Note**
> These endpoints are called by interactive buttons in prompt messages. They are not intended to be called directly.

POST /api/v1/prompt/accept TEAM ADMIN+

Accept button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."}
```

Links inbound connection to team, deletes the prompt, and updates the prompt post message to show acceptance. Returns a `PostActionIntegrationResponse`.

POST /api/v1/prompt/block TEAM ADMIN+

Block button on team-level inbound connection prompt.

Context

```
{"team_id": "...", "conn_name": "..."}
```

Sets prompt state to `blocked` and updates the prompt post message. Connection stays unlinked until `reset-prompt` is used.

POST **/api/v1/prompt/channel/accept** TEAM ADMIN+

Accept button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."}
```

Links inbound connection to channel, deletes prompt, and updates the post.

POST **/api/v1/prompt/channel/block** TEAM ADMIN+

Block button on channel-level inbound connection prompt.

Context

```
{"channel_id": "...", "conn_name": "..."}
```

Sets channel prompt state to `blocked` and updates the post.

Connection Requests

> **Note**
> These endpoints are called by interactive buttons in connection request DMs sent to System Admins. They are not intended to be called directly. Requests are created when a Team Admin runs

> `/crossguard init-team` (or the REST equivalent) while the plugin is in request mode (`RestrictToSystemAdmins` + `AllowTeamAdminRequests` both enabled).

POST `/api/v1/request/approve` SYSTEM ADMIN

Approve a pending connection link request. Links the connection to the team, deletes the request, updates all admin DM posts, and notifies the requester.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."}
```

Behavior

- Executes the team init (same as a direct System Admin link)
- Removes Approve/Deny buttons from all admin DM posts for this request
- Sends DM to requester confirming approval
- If the connection was removed from config while pending, cancels the request and notifies the requester

Returns a `PostActionIntegrationResponse`.

POST `/api/v1/request/deny` SYSTEM ADMIN

Deny a pending connection link request. Opens an interactive dialog for the admin to provide an optional reason.

Request Body

`PostActionIntegrationRequest` with context:

```
{"team_id": "...", "conn_key": "..."}
```

Behavior

- Opens a dialog with an optional "Reason for denial" text area
- Dialog submission is handled by `/api/v1/request/deny-submit`

Returns a `PostActionIntegrationResponse`.

POST `/api/v1/request/deny-submit` SYSTEM ADMIN

Dialog submission handler for deny with optional reason.

Request Body

`SubmitDialogRequest` with:

- `state`: `team_id|conn_key`
- `submission.reason`: optional denial reason text

Behavior

- Deletes the pending request
- Removes Approve/Deny buttons from all admin DM posts for this request
- Sends DM to requester with denial and optional reason
- If the dialog was cancelled, does nothing

Returns `200` on success.

Channel Connection Requests

When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can submit channel-level connection link requests. These endpoints handle the Team Admin approval workflow. They mirror the team-level request endpoints but operate on channel connections with Team Admins as the approvers.

POST /api/v1/channel-request/approve TEAM ADMIN+

Approve a pending channel connection link request. Links the connection to the channel, deletes the request, updates all Team Admin DM posts, and notifies the requester.

Request Body

`PostActionIntegrationRequest` with context:

```
{"channel_id": "...", "team_id": "...", "conn_key": "..."}
```

Behavior

- Validates the connection still exists in config and the team still has it linked
- Validates the target channel still exists
- Executes the channel init (same as a direct Channel Admin link)
- Removes Approve/Deny buttons from all Team Admin DM posts for this request
- Sends DM to requester confirming approval
- If the connection was removed from config or unlinked from the team while pending, cancels the request and notifies the requester

Returns a `PostActionIntegrationResponse`.

POST /api/v1/channel-request/deny TEAM ADMIN+

Deny a pending channel connection link request. Opens an interactive dialog for the Team Admin to provide an optional reason.

Request Body

`PostActionIntegrationRequest` with context:

```
{"channel_id": "...", "team_id": "...", "conn_key": "..."}
```

Behavior

- Opens a dialog with an optional "Reason for denial" text area
- Dialog submission is handled by `/api/v1/channel-request/deny-submit`

Returns a `PostActionIntegrationResponse`.

POST `/api/v1/channel-request/deny-submit` TEAM ADMIN+

Dialog submission handler for channel request deny with optional reason.

Request Body

`SubmitDialogRequest` with:

- `state`: `channel_id|team_id|conn_key`
- `submission.reason`: optional denial reason text

Behavior

- Deletes the pending channel connection request
- Removes Approve/Deny buttons from all Team Admin DM posts for this request
- Sends DM to requester with denial and optional reason
- If the dialog was cancelled, does nothing

Returns `200` on success.

Dialog Handling

POST `/api/v1/dialog/select-connection` AUTHENTICATED

Interactive dialog handler when user selects a connection from the dropdown.

Request Body

`SubmitDialogRequest` with:

- `connection_name` field: selected connection in `direction:name` format
- `state` field: `action:target_id` (e.g., `init-team:abc123`, `teardown-channel:xyz789`)

Supported Actions

Action	Description
`init-team`	Link connection to team
`teardown-team`	Unlink connection from team
`init-channel`	Link connection to channel
`teardown-channel`	Unlink connection from channel

Validates permissions, performs the action, and sends an ephemeral message with the result. Returns `200` on success.

Message Transport Interface

Cross Guard supports NATS, Azure Queue Storage, Azure Blob Storage, and Azure Service Bus as transport layers for relaying messages between Mattermost instances. For the complete wire protocol reference, including all message envelope schemas, payload definitions, connection configuration, data flow diagrams, and full wire format examples, see the dedicated Transport Interface Reference.

Error Codes

Reference for every numeric `error_code` emitted by Cross Guard logs

How to Read This Page

Every `p.API.Log*` call in the Cross Guard plugin (non-test code) carries a stable numeric `error_code` as its first key-value pair. Operators can grep production logs for the integer value even if the Go constant name changes later. Numbers are never reused or renumbered once assigned.

Example log line:

```
{"level":"error","msg":"Failed to check channel connections","error_code":10000,"chann
```

Codes are allocated per source file in non-overlapping 1000-wide ranges. The first two digits identify the subsystem. Each row below lists the exact log message, the fields you can expect alongside `error_code`, and the level at which the message is emitted (`DEBUG` , `INFO` , `WARN` , `ERROR`).

> **Informational and warning codes**
> Not every code represents a failure. Some mark successful events (connection established, subscription started, retry succeeded) or expected drops (policy filters, max retries reached). Check the log level first: `INFO` lines never require action, `WARN` lines are usually recoverable, and `ERROR` lines indicate a broken operation.

Range Quick Reference

Range	Source file	Subsystem	Count
`10000-10999`	hooks.go	Mattermost post and reaction event hooks	12

Range	Source file	Subsystem	Count
`11000-11999`	api.go	REST API handlers	17
`12000-12999`	configuration.go	Plugin configuration lifecycle	3
`13000-13999`	command.go	Slash command handlers	1
`14000-14999`	service.go	Team and channel initialization service	34
`15000-15999`	inbound.go	Inbound message dispatch and file watcher	46
`16000-16999`	connections.go	Outbound publisher and file upload	14
`17000-17999`	sync_user.go	Synthetic relay-user resolution	4
`18000-18999`	azure_blob_provider.go	Azure Blob WAL provider	50
`19000-19999`	azure_provider.go	Azure Queue provider	11
`20000-20999`	nats_provider.go	NATS transport provider	4
`21000-21999`	prompt.go	Inbound connection prompts	17
`22000-22999`	retry_dispatch.go	Missing-parent post retry queue	5
`23000-23999`	store/caching.go	Cluster cache invalidation	1

Range	Source file	Subsystem	Count
`24000-24999`	request.go	Team admin connection request workflow	17
`25000-25999`	channel_request.go	Channel admin connection request workflow	18
`26000-26999`	azure_servicebus_provider.go	Azure Service Bus provider	8

hooks.go (10000-10999)

Emitted from Mattermost post and reaction lifecycle hooks (`MessageHasBeenPosted`, `MessageHasBeenUpdated`, `MessageHasBeenDeleted`, `ReactionHasBeenAdded`, `ReactionHasBeenRemoved`). These run on every message event on the server, so transient failures here can prevent relay of individual messages.

Code	Name / Level	Log message and context	Troubleshooting
`10000`	`HooksChannelConnCheckFailed` **ERROR**	"Failed to check channel connections" with `channel_id`. The post hook tried to look up which Cross Guard connections are linked to the channel where a message was just posted, and the KV store lookup failed.	Check the `error` field for the underlying KV store error. Usually transient (store restart, read pressure). If repeat for the same `channel_id`, run `/crossguard status` in that channel to compare what the store returns. If the store corrupt, remove an re-add the channel link with `teardown`

Code	Name / Level	Log message and context	Troubleshooting
			`channel` then `init-channel`.
`10001`	`HooksGetChannelFailed` **ERROR**	"Failed to get channel for relay" with `channel_id`. Mattermost returned an error when the hook asked for the channel record by ID.	Channel may have been deleted between the event and the hook firing, which is safe to ignore. If persistent check Mattermost database health an that the plugin has read-channel permission. Grep b `channel_id` to s whether one specif channel is affected
`10002`	`HooksTeamConnCheckFailed` **ERROR**	"Failed to check team connections" with `team_id`. The hook tried to read the team-level connection list before relaying a message.	Same pattern as 10000: inspect the adjacent `error` field. The team ma have a partial or corrupt link. Runnir `/crossguard status` in the tea confirms whether t link is readable.
`10003`	`HooksGetTeamFailed` **ERROR**	"Failed to get team for relay" with `team_id`. Mattermost returned an error when the hook asked for the team record.	Team may have be deleted, or Mattermost is unde load. Check databa health. Transient single occurrences are safe to ignore.
`10004`	`HooksRelaySemaphoreFull` **WARN**	"Relay semaphore full, dropping event" with `context`. The per-process	Raise `RelayConcurren` in System Console

Code	Name / Level	Log message and context	Troubleshooting
		relay concurrency semaphore was saturated when a new event arrived, so the message was dropped rather than blocking the hook.	Plugins > Cross Guard. Look for correlated slownes in `connections.g` logs (16001, 16006 16007): the downstream provid is likely slow or unreachable. Sustained semaphore-full events mean the plugin cannot keep up with the local message rate.
`10005`	`HooksGetUserForPostFailed` **ERROR**	"Failed to get user for relay" with `user_id`. The create-post hook failed to load the author before serializing the envelope.	User may have bee deleted, or the plug lacks read-user permission. Check the `error` field. Occasional failures are safe to ignore because the next message from the same user will retry
`10006`	`HooksGetUserForUpdateFailed` **ERROR**	"Failed to get user for relay" with `user_id`. Same as 10005, but from the edit-post hook.	Same as 10005.
`10007`	`HooksDeleteFlagCheckFailed` **ERROR**	"Failed to check delete flag, skipping relay to avoid loop" with `post_id`. The delete hook could not check the `crossguard_delete_flag` used to break relay loops, so it skipped relaying this delete.	Inspect the KV stor error. A persistent failure means delet events stop propagating. Check that the KV store is reachable and not of quota. The flag h

Code	Name / Level	Log message and context	Troubleshooting
			a short TTL, so transient failures se heal.
`10008`	`HooksGetPostForReactAddFailed` **ERROR**	"Failed to get post for reaction relay" with `post_id`. The reaction-add hook could not load the target post.	Post may have bee deleted concurrentl Safe to ignore sing instances. If repeated, check Mattermost post st health.
`10009`	`HooksGetUserForReactAddFailed` **ERROR**	"Failed to get user for reaction relay" with `user_id`. Reaction-add hook could not load the reacting user.	Same as 10005.
`10010`	`HooksGetPostForReactRemFailed` **ERROR**	"Failed to get post for reaction relay" with `post_id`. Reaction-remove hook could not load the target post.	Same as 10008.
`10011`	`HooksGetUserForReactRemFailed` **ERROR**	"Failed to get user for reaction relay" with `user_id`. Reaction-remove hook could not load the user whose reaction was removed.	Same as 10005.

api.go (11000-11999)

Emitted from REST API handlers. Most codes correspond to a System Console action (test connection, set rewrite) or to a slash-command action dispatched via the API.

Code	Name / Level	Log message and context	Troubleshooting
`11000`	`APINATSTestConnectFailed` **ERROR**	"NATS connection test failed" with `address`. The `/test-connection` handler could not dial the NATS broker with the supplied configuration.	Verify network reachabi settings, and auth crede user/password) in the c the plugin host, try `tel` or `openssl s_client`
`11001`	`APIBuildTestMessageFailed` **ERROR**	"Failed to build test message". The test-connection path could not construct the test envelope.	Internal bug. Capture th field and file an issue. F unless the plugin is upd
`11002`	`APIPublishTestMessageFailed` **ERROR**	"Failed to publish test message" with `subject`. Dialing succeeded but the publish call failed.	Verify that the NATS ac permission on the confi that the subject starts w Check broker-side logs
`11003`	`APIFlushTestMessageFailed` **ERROR**	"Failed to flush test message" with `subject`. Publish was buffered but flush (forcing delivery) failed, usually because the connection dropped.	Retry the test. If reprod closing connections sho Check TLS hostname v keepalive settings, and
`11004`	`APITestMessageSent` **INFO**	"Test message sent" with `subject`, `msg_id`. Informational confirmation that the outbound test envelope was published.	Informational. The `msg` with `InboundTestMes` (15011) on the receivin

Code	Name / Level	Log message and context	Troubleshooting
`11005`	`APISubscribeInboundTestFailed` **ERROR**	"Failed to subscribe for inbound test" with `subject`. The inbound test-connection path could not subscribe to the NATS subject.	Verify the account has s on the subject. Multi-ter are often scoped to spe
`11006`	`APIFlushNATSConnFailed` **ERROR**	"Failed to flush NATS connection". Cleanup flush failed on the short-lived test client.	Non-fatal if the primary success. Indicates the t early.
`11007`	`APIInboundTestSubscribeOK` **INFO**	"Inbound test subscription successful" with `subject`. Informational.	Informational.
`11008`	`APIAzureQueueTestFailed` **ERROR**	"Azure Queue connection test failed". The Azure Queue test could not reach or authenticate against the configured queue endpoint.	Check `queue_servic` `account_name`, `acc` `queue_name`. For Azu emulator is running and Azurite format (`http://host:10001` For production, confirm the account key has no
`11009`	`APIAzureBlobTestFailed` **ERROR**	"Azure Blob connection test failed". The Azure Blob test could not reach or authenticate against the configured blob container.	Check `service_url`, `blob_container_nam` container already exists can create it. For Azurit emulator is running on i See Admin Configuratio setup.
`11010`	`APIGetUserFailed` **ERROR**	"Failed to get user" with `user_id`. An API handler could not	Usually means an expir transient store error. Ha

Code	Name / Level	Log message and context	Troubleshooting
		load the requesting user from Mattermost.	webapp. If repeated for Mattermost store health
`11011`	`APIInitChannelGetTeamConns` **ERROR**	"Failed to get team connections" with `team_id`. Channel-init endpoint could not load the owning team's connections (needed to auto-init the team).	Confirm the team exists `init-team` explicitly fi store error.
`11012`	`APITeardownChannelGetChanConns` **ERROR**	"Failed to get channel connections" with `channel_id`. Channel-teardown endpoint could not load the channel's current connections.	Inspect the KV store err
`11013`	`APITeardownTeamGetTeamConns` **ERROR**	"Failed to get team connections" with `team_id`. Team-teardown endpoint could not load the team's connections.	Inspect the KV store err
`11014`	`APIBulkChannelConnectionsFailed` **WARN**	"Failed to get channel connections for bulk lookup" with `channel_id`. The bulk channel connection lookup used by the webapp channel indicator failed for one channel in the batch.	Single failures only affe are safe to ignore. Repe many channels mean th The rest of the batch is

Code	Name / Level	Log message and context	Troubleshooting
`11015`	`APITeamRewriteSet` **INFO**	"Team rewrite set" with `team_id`, `connection`, `remote_team_name`, `user`. Audit log of a successful team name rewrite.	Informational and audit· field is the admin who s
`11016`	`APITeamRewriteCleared` **INFO**	"Team rewrite cleared" with `team_id`, `connection`, `user`. Audit log of a cleared rewrite.	Informational and audit·

configuration.go (12000-12999)

Emitted from plugin configuration load and validation.

Code	Name / Level	Log message and context	Troubleshooting
`12000`	`ConfigSameConfigPassed` **WARN**	"setConfiguration called with the existing configuration". The `OnConfigurationChange` hook fired but Mattermost passed the same config object that is already loaded.	Usually benign, fired during plugin activation and config reloads. Spammy occurrences suggest a tight loop calling `SavePluginConfig`. Check for misbehaving admin integrations.
`12001`	`ConfigValidationWarn` **WARN**	"Plugin configuration has validation warnings". Non-fatal validation warnings were raised while parsing plugin settings.	Read the warning text in the adjacent `warnings` field (connection with obsolete field, duplicate name, etc.). Edit the

Code	Name / Level	Log message and context	Troubleshooting
			connection in System Console > Plugins > Cross Guard and save again to clear.
`12010`	`AzureSPAuditConstructed` **INFO**	"Azure auth (service-principal): credential constructed" with `provider` (one of `azure-queue`, `azure-blob`, `azure-servicebus`), `auth_mode` (always `service-principal`), `tenant_id`, `client_id`, `azure_cloud`, and `secret_hash_prefix` (first 12 hex chars of SHA-256 of the resolved secret). Emitted once per Service Principal credential construction so operators have a record of which AAD identity authenticated each connection. The secret value itself is never logged.	Audit-only; not an error. Use `secret_hash_prefix` to correlate which secret was active during an incident without exposing material that could be used to reconstruct it.

command.go (13000-13999)

Emitted from slash command handlers.

Code	Name / Level	Log message and context	Troubleshooting
`13000`	`CommandOpenConnDialogFailed` **ERROR**	"Failed to open connection dialog". The user ran a slash command (e.g.,	Verify the Mattermost site URL is set correctly and that the server can reach its own plugin dialog

Code	Name / Level	Log message and context	Troubleshooting
		`/crossguard init-team`) without arguments and the plugin failed to open the interactive selection dialog.	endpoint. Check the `error` field for the specific RPC failure. Users can work around by passing the connection name directly on the command line.

service.go (14000-14999)

Emitted from the team and channel initialization service. This layer persists the team-to-connection and channel-to-connection links in the KV store, writes init and teardown announcement posts, and enforces header prefixes on linked channels.

Code	Name / Level	Log message and context	Troubleshooting
`14000`	`ServiceInitTeamGetConnsFailed` **ERROR**	"Failed to get team connections" with `team_id`. `init-team` could not read the team's existing connection list before adding a new one.	Inspect the KV store Usually a store outag restored database w plugin tables. The ini is safely idempotent; the store is healthy.
`14001`	`ServiceAddTeamConnFailed` **ERROR**	"Failed to add team connection" with `team_id`, `conn`. `init-team` failed to write the new team-to-connection link.	Inspect the KV store after the store recove operation partially wr `init-team` again i

Code	Name / Level	Log message and context	Troubleshooting
`14002`	`ServiceInitTeamReReadConnsFailed` **ERROR**	"Failed to re-read team connections" with `team_id`. After writing the link, the service could not re-read the team's connections to confirm the update.	The write may still ha succeeded. Run `/c` `status` in the team `/api/v1/teams/<i` to verify.
`14003`	`ServiceAddTeamInitializedFailed` **ERROR**	"Failed to add team to initialized list" with `team_id`. Failed to update the "initialized teams" index used by global status.	Inspect the KV store primary link is still in global status may mi Re-running `init-t` the index.
`14004`	`ServicePostTeamInitMsgFailed` **WARN**	"Failed to post initialization message". The service successfully linked the team but failed to post the confirmation message in town-square.	Non-fatal. Confirm th a town-square chanr and that the Cross G post permission. Tea initialization is still co

Code	Name / Level	Log message and context	Troubleshooting
`14005`	`ServiceCheckTeamStatusFailed` **ERROR**	"Failed to check team status" with `team_id`. Internal status lookup failed while the service was answering a status query.	Inspect the KV store Affects `/crossguar` output.
`14006`	`ServiceGetInitializedTeamsFailed` **ERROR**	"Failed to get initialized teams". Global status failed to load the list of initialized teams.	Inspect the KV store Affects `GET /api/v`
`14007`	`ServiceTeamStatusLookupTeamFailed` **WARN**	"Failed to look up team for status" with `team_id`. Status endpoint failed to load a team record.	Team may have bee since initialization. R `teardown-team` to manually remove the
`14008`	`ServiceTeamStatusGetConnsFailed` **WARN**	"Failed to get team connections for status" with `team_id`. Status endpoint failed to load the team's connections.	Inspect the KV store
`14009`	`ServiceParseOutboundConnFailed` **WARN**	"Failed to parse outbound connection configuration". A stored team link references an outbound connection that could not be	The connection was deleted in System C being linked. The link as `orphaned: tru` output. Run `teardo` with the old name, o connection in Systen

Code	Name / Level	Log message and context	Troubleshooting
		parsed from the current plugin config.	
`14010`	`ServiceParseInboundConnFailed` **WARN**	"Failed to parse inbound connection configuration". Same as 14009, but for inbound connections.	Same as 14009.
`14011`	`ServiceTeardownGetChanConnsFailed` **ERROR**	"Failed to get channel connections" with `channel_id`. During team teardown, failed to list channel connections nested under the team.	Inspect the KV store Orphaned channel li remain and can be c with explicit `teardo` `channel` calls.
`14012`	`ServiceTeardownGetTeamConnsFailed` **ERROR**	"Failed to get team connections" with `team_id`. Team teardown failed to load the team's own connection list.	Inspect the KV store
`14013`	`ServiceInitChanGetTeamConnsFailed` **ERROR**	"Failed to get team connections" with `team_id`. `init-channel` could not verify the parent team's connections.	Confirm the team is i (`/crossguard sta` run `init-team` firs

Code	Name / Level	Log message and context	Troubleshooting
`14014`	`ServiceInitChanGetChanConnsFailed` **ERROR**	"Failed to get channel connections" with `channel_id`. `init-channel` could not read the channel's existing connections.	Inspect the KV store
`14015`	`ServiceAddChanConnFailed` **ERROR**	"Failed to add channel connection" with `channel_id`, `conn`. Failed to write a new channel-to-connection link.	Inspect the KV store command is idempot
`14016`	`ServiceInitChanReReadConnsFailed` **ERROR**	"Failed to re-read channel connections" with `channel_id`. After linking, re-read failed.	Verify with `/crossg` `status` in the chan
`14017`	`ServiceChanHeaderPrefixFailed` **WARN**	"Failed to update channel header with CrossGuard prefix" with `channel_id`. The channel was linked but the header could not be updated to add the Cross Guard marker.	Verify the bot has ma channel permission c channel. The link itse active; only the visibl missing.

Code	Name / Level	Log message and context	Troubleshooting
`14018`	`ServicePostChanInitMsgFailed` **WARN**	"Failed to post channel init message". Channel was linked but the confirmation message could not be posted.	Confirm the bot can target channel. Non-
`14019`	`ServiceRemoveChanGetConnsFailed` **ERROR**	"Failed to get channel connections" with `channel_id`. `teardown-channel` could not load the current connections.	Inspect the KV store
`14020`	`ServiceRemoveChanConnFailed` **ERROR**	"Failed to remove channel connection" with `channel_id`, `conn`. Failed to delete a single channel-to-connection link.	Inspect the KV store retry. Orphaned links removed by re-runni `teardown-channel`
`14021`	`ServiceRemoveChanReReadConnsFailed` **ERROR**	"Failed to re-read channel connections" with `channel_id`. Post-removal verify read failed.	Verify with `/crossg status`.
`14022`	`ServiceDeleteChanConnsFailed` **ERROR**	"Failed to delete channel connections" with `channel_id`.	Inspect the KV store retry teardown.

Code	Name / Level	Log message and context	Troubleshooting
		Failed to delete the channel's full connection record during teardown.	
`14023`	`ServiceChanHeaderRemovePrefixFailed` **WARN**	"Failed to remove CrossGuard prefix from channel header" with `channel_id`. Could not strip the Cross Guard marker from the channel header during teardown.	Verify the bot has ma channel permission. edit the header manu remove the leftover r
`14024`	`ServicePostChanTeardownMsgFailed` **WARN**	"Failed to post channel teardown message". Could not post the "channel unlinked" confirmation message.	Non-fatal. Confirm b
`14025`	`ServiceTeardownTeamGetConnsFailed` **ERROR**	"Failed to get team connections" with `team_id`. Team teardown failed to load connections during cleanup.	Inspect the KV store
`14026`	`ServiceRemoveTeamConnFailed` **ERROR**	"Failed to remove team connection" with `team_id`, `conn`. Failed to delete a team-to-connection link during teardown.	Inspect the KV store retry.

Code	Name / Level	Log message and context	Troubleshooting
`14027`	`ServiceTeardownTeamReReadConnsFailed` **ERROR**	"Failed to re-read team connections" with `team_id`. Post-removal verify read failed.	Verify with `/crossg` `status`.
`14028`	`ServiceRemoveTeamInitializedFailed` **ERROR**	"Failed to remove team from initialized list" with `team_id`. Failed to update the "initialized teams" index during teardown.	Inspect the KV store status may still list th re-run of teardown.
`14029`	`ServicePostTeamTeardownMsgFailed` **WARN**	"Failed to post team teardown message". Non-fatal.	Confirm bot can post square.
`14030`	`ServiceGlobalParseOutConnFailed` **WARN**	"Failed to parse outbound connections". Global status could not parse an outbound connection from plugin config when answering `GET /api/v1/status`.	Review the outbound form in System Cons adjacent `error` fie specific parse failure
`14031`	`ServiceGlobalParseInConnFailed` **WARN**	"Failed to parse inbound connections". Same as 14030 for inbound connections.	Same as 14030.

Code	Name / Level	Log message and context	Troubleshooting
`14032`	`ServiceMapParseOutConnFailed` **WARN**	"Failed to parse outbound connections for map". Startup failed to build the internal outbound connection map.	Inspect the precedin error. The failing con be skipped from the until fixed.
`14033`	`ServiceMapParseInConnFailed` **WARN**	"Failed to parse inbound connections for map". Same as 14032 for inbound connections.	Same as 14032.

inbound.go (15000-15999)

Emitted from the inbound dispatcher that consumes messages from the transport, applies idempotency, and materializes posts, edits, deletes, and reactions locally. Also covers the inbound file watcher loop.

Code	Name / Level	Log message and context	Troublesho
`15000`	`InboundParseConnsFailed` **ERROR**	"Failed to parse inbound connections". Startup failed to parse any inbound connections from plugin config.	Review all i connections Console. Fi flagged in tl `error` fie least one pa inbound co messages received.
`15001`	`InboundConnectFailed` **ERROR**	"Failed to connect inbound" with `name`, `provider`. An inbound connection failed to	Check prov codes in the context (18

Code	Name / Level	Log message and context	Troublesho
		dial its transport (NATS, Azure Queue, or Azure Blob).	20xxx). Ver reachability and subject/que names. The will retry on interval.
`15002`	`InboundSubscribeFailed` **ERROR**	"Failed to subscribe inbound" with `name`. Connected to NATS but subscribe call failed.	Verify the N has subscri on the subj broker-side
`15003`	`InboundSubscriptionEstablished` **INFO**	"Inbound subscription established" with `name`, `provider`. Informational confirmation of a successful inbound subscription.	Information liveness sig plugin resta
`15004`	`InboundRelaySemaphoreFull` **WARN**	"Relay semaphore full, dropping inbound message" with `conn`. The inbound concurrency semaphore was saturated; the current message was dropped.	Raise `RelayCon` investigate slowness: s `CreatePo` database la hooks on th Sustained d the receiver up.
`15005`	`InboundUnmarshalFailed` **ERROR**	"Failed to unmarshal inbound message" with `conn`. Received an envelope that parsed neither as JSON nor XML.	Check that uses a com version and `message_` matches on Capture the for diagnosi
`15006`	`InboundPostMissingPayload` **ERROR**	"Inbound post: missing payload" with `conn`. A	Sender bug corruption i

Code	Name / Level	Log message and context	Troublesho
		`post` envelope arrived with no payload section.	Capture the confirm sen and file an i
`15007`	`InboundUpdateMissingPayload` **ERROR**	"Inbound update: missing payload" with `conn`. A `post_update` envelope arrived with no payload.	Same as 15
`15008`	`InboundDeleteMissingPayload` **ERROR**	"Inbound delete: missing payload" with `conn`. A `post_delete` envelope arrived with no payload.	Same as 15
`15009`	`InboundReactionAddMissingPayload` **ERROR**	"Inbound reaction add: missing payload" with `conn`. A `reaction_add` envelope arrived with no payload.	Same as 15
`15010`	`InboundReactionRemoveMissingPayload` **ERROR**	"Inbound reaction remove: missing payload" with `conn`. A `reaction_remove` envelope arrived with no payload.	Same as 15
`15011`	`InboundTestMessageReceivedWithID` **INFO**	"Received inbound test message" with `conn`, `id`. The inbound test subscriber received a test envelope with an ID.	Information correlates t `APITestM` (11004) on
`15012`	`InboundTestMessageReceived` **INFO**	"Received inbound test message" with `conn`. Test envelope without an ID.	Information
`15013`	`InboundUnknownMessageType` **WARN**	"Unknown inbound message type" with `conn`, `type`. An envelope with an	Usually me sender is ta older receiv

Code	Name / Level	Log message and context	Troublesho
		unrecognized `type` field arrived.	the receiver Check the s version.
`15014`	`InboundMissingMessageQueueFull` **ERROR**	"Missing message: queue full, dropping message" with `conn`, `type`, `remote_post_id`, `queue_size`. The retry queue for messages whose parent post was missing locally was full; the new message was dropped.	Increase th parent queu settings, or why parent arriving. Se retry_dispat how the que entries. A p queue usua sender neve the parent p
`15015`	`InboundMissingMessageQueued` **WARN**	"Missing message: queuing for retry" with `conn`, `type`, `remote_post_id`, `queue_size`. A message arrived whose parent post has not yet been received; it was queued for retry.	Normal duri traffic or out delivery. Th dispatcher v (see 22000 Rising `que` across man indicates or problems in
`15016`	`InboundPostResolveFailed` **WARN**	"Inbound post: resolve failed" with `conn`. Failed to resolve the target team/channel for an inbound post.	Most comm the remote does not ma team and n configured. `/crossgua` `rewrite-t` Slash Comm ensure tear match on b
`15017`	`InboundPostIdempotencyLookupFailed` **WARN**	"Inbound post: idempotency lookup failed, processing anyway" with `conn`,	Inspect the error. Risk: duplicate po

Code	Name / Level	Log message and context	Troublesho
		`postID`. The idempotency cache lookup failed, so the plugin cannot guarantee this post was not already created; it proceeds anyway.	message is by the trans
`15018`	`InboundPostResolveUserFailed` **ERROR**	"Inbound post: resolve user failed" with `conn`, `username`. Failed to resolve the post author to a local or sync user.	Check `Username` setting and user creatio for this use Correlated codes will s underlying f post is drop
`15019`	`InboundPostRootMappingLookupFailed` **WARN**	"Inbound post: failed to look up root mapping" with `conn`, `remote_root_id`. A thread reply arrived but the root-post mapping lookup failed.	Inspect the error.
`15020`	`InboundPostRootNotFoundStandalone` **WARN**	"Inbound post: root not found after retries, creating standalone" with `conn`, `remote_root_id`. After the retry dispatcher gave up waiting for the parent, the plugin created the reply as a standalone post at the top level of the channel.	Indicates th never arrive may have fi missed, or f it). Check th for the `remote_r` Consider ra dispatcher's slower root- not missed.
`15021`	`InboundPostCreateFailed` **ERROR**	"Inbound post: create failed" with `conn`. Mattermost rejected the `CreatePost` call.	Common ca length over characters, archived or lacks post p

Code	Name / Level	Log message and context	Troublesho
			server-side error. Inspe field.
`15022`	`InboundPostStoreMappingFailed` **ERROR**	"Inbound post: failed to store post mapping" with `conn`, `remote_id`, `local_id`. The post was created locally, but saving the remote-to-local mapping failed.	Inspect the error. Futur deletes of t not find the Manual clea needed; co if this recurs
`15023`	`InboundUpdateMappingLookupFailed` **ERROR**	"Inbound update: failed to look up post mapping" with `conn`, `remote_id`. Could not find the local post ID for an incoming edit.	Possible ca original pos reached thi (check 150 the mappin (15027-150 is dropped.
`15024`	`InboundUpdateGetLocalPostFailed` **ERROR**	"Inbound update: failed to get local post" with `conn`, `local_id`. Mapping existed but Mattermost could not load the local post.	Post may h deleted loca ignore singl
`15025`	`InboundUpdatePostFailed` **ERROR**	"Inbound update: failed to update post" with `conn`, `local_id`. Mattermost rejected the `UpdatePost` call.	Common ca deleted con validation e edit permiss the `error`
`15026`	`InboundDeleteMappingLookupFailed` **ERROR**	"Inbound delete: failed to look up post mapping" with `conn`, `remote_id`. Could not find the local post ID for an incoming delete.	Same as 15
`15027`	`InboundDeleteSetFlagFailed` **ERROR**	"Inbound delete: failed to set delete flag" with `conn`,	Inspect the error. Witho

Code	Name / Level	Log message and context	Troublesho
		`local_id`. Failed to set the `crossguard_delete_flag` that prevents the delete from echoing back through the outbound hook.	the delete r back to the creating a l 10007 on th side.
`15028`	`InboundDeletePostFailed` **ERROR**	"Inbound delete: failed to delete post" with `conn`, `local_id`. Mattermost rejected the `DeletePost` call.	Post may a deleted. Ins `error` fie
`15029`	`InboundDeleteRemoveFlagFailed` **WARN**	"Inbound delete: failed to remove delete flag" with `conn`, `local_id`. Failed to clear the delete flag after a successful delete.	Non-fatal. T TTL and ex automatical
`15030`	`InboundDeleteRemoveMappingFailed` **WARN**	"Inbound delete: failed to remove post mapping" with `conn`, `remote_id`. Leaves a stale mapping entry.	Inspect the error. Stale only a spac does not af correctness
`15031`	`InboundReactionMappingLookupFailed` **ERROR**	"Inbound reaction: failed to look up post mapping" with `conn`, `remote_id`. Could not find the local post for a reaction event.	Same as 15
`15032`	`InboundReactionResolveFailed` **WARN**	"Inbound reaction: resolve failed" with `conn`. Failed to resolve the target channel for a reaction.	Same as 15
`15033`	`InboundReactionResolveUserFailed` **ERROR**	"Inbound reaction: resolve user failed" with `conn`, `username`. Failed to resolve the reacting user.	Same as 15

Code	Name / Level	Log message and context	Troublesho
`15034`	`InboundReactionAddFailed` **ERROR**	"Inbound reaction: add failed" with `conn`, `post_id`. Mattermost rejected the `AddReaction` call.	Post may n or the bot la permission. `error` fie
`15035`	`InboundReactionRemoveFailed` **ERROR**	"Inbound reaction: remove failed" with `conn`, `post_id`. Mattermost rejected the `RemoveReaction` call.	Same as 15
`15036`	`InboundFileWatcherStarted` **INFO**	"File watcher started" with `conn`. The inbound file watcher goroutine started.	Information at plugin sta inbound co file transfer
`15037`	`InboundFileWatcherExited` **ERROR**	"File watcher exited with error" with `conn`. The watcher loop exited, usually due to a fatal provider error.	Check the a `error` fie correlated `18xxx` / `1` codes. The restarted or tick. Persist indicate cre network out
`15038`	`InboundFileMissingHeaders` **WARN**	"Inbound file: missing required headers, skipping" with `key`. A staged file blob lacked the expected Cross Guard metadata headers.	Something into the Cro container w envelope h Usually stal Capture the `key` for di

Code	Name / Level	Log message and context	Troublesho
`15039`	`InboundFileConnInactive` **WARN**	"Inbound file: connection no longer active, skipping" with `conn`, `filename`. A file arrived for a connection that is no longer active on this instance.	Expected in after a `tea` Persistent c the sender to a torn-do connection.
`15040`	`InboundFileFilteredByPolicy` **INFO**	"Inbound file filtered by policy" with `filename`, `conn`. A file was rejected by the connection's allow/deny extension filter.	Expected w configured. `file_filt` `file_filt` the connect was wrong.
`15041`	`InboundFileMappingLookupFailed` **ERROR**	"Inbound file: post mapping lookup failed after retries" with `conn`, `remote_post_id`. Tried to find the local post ID for the file's parent post but the lookup kept failing.	Inspect the error. The fi attached, a left in place (Azure) or c (NATS).
`15042`	`InboundFileNoMappingFound` **WARN**	"Inbound file: no post mapping found after retries" with `conn`, `remote_post_id`. The mapping lookup succeeded (no store error) but no local post exists for the remote ID.	The parent arrived or w out. Check for the `remote_po` Usually tran out-of-order
`15043`	`InboundFileGetLocalPostFailed` **ERROR**	"Inbound file: failed to get local post" with `local_post_id`. Mapping existed but Mattermost could not load the local post.	Post may h deleted. Sa
`15044`	`InboundFileUploadFailed` **ERROR**	"Failed to upload file to Mattermost" with `filename`. Mattermost	Check Matt `MaxFileSi` storage quo

Code	Name / Level	Log message and context	Troublesho
		rejected the `UploadFile` call.	available di `error` fie
`15045`	`InboundFileAttachFailed` **ERROR**	"Failed to attach file to post" with `local_post_id`, `file_id`. File was uploaded but the association with the post failed.	Usually a p invalid `pos` Inspect the

connections.go (16000-16999)

Emitted from the outbound publisher pipeline: envelope serialization, multi-part splitting, provider publish with retries, and file upload to object store.

Code	Name / Level	Log message and context	Troubleshooting
`16000`	`ConnectionsParseOutboundFailed` **ERROR**	"Failed to parse outbound connections for relay". Failed to parse outbound connections at publish time.	Inspect the adjacent `error` field and fix the connection form in System Console. See also 14030.
`16001`	`ConnectionsConnectOutboundFailed` **ERROR**	"Failed to connect outbound for relay" with `name`, `provider`. Outbound provider dial failed when a message needed to be published.	Verify provider reachability and credentials. Check correlated `18xxx` / `19xxx` / `20xxx` codes. Publish will retry per `PublishRetries` before giving up (16007)

Code	Name / Level	Log message and context	Troubleshooting
`16002`	`ConnectionsOutboundEstablished` **INFO**	"Outbound connection established for relay" with `name`, `provider`. Informational confirmation.	Informational. Useful as a liveness signal.
`16003`	`ConnectionsSerializeFailed` **ERROR**	"Failed to serialize outbound message" with `name`, `format`. Could not serialize the envelope to JSON or XML.	Internal bug. Capture the adjacent `error` field and the source post ID for diagnosis.
`16004`	`ConnectionsMessageSplit` **INFO**	"Message split into parts for provider size limit" with `connection`, `parts`, `originalSize`. A large message was split into multiple transport parts.	Informational. Frequent splits with high `parts` counts indicate very large posts; consider whether file transfer would be more efficient.
`16005`	`ConnectionsSerializePartFailed` **ERROR**	"Failed to serialize message part" with `name`, `part`. Part of a split message failed to serialize.	Same as 16003.

Code	Name / Level	Log message and context	Troubleshooting
`16006`	`ConnectionsPublishPartFailed` **ERROR**	"Failed to publish message part" with `name`, `part`. Publish of a single part failed on one attempt.	Transient if followed by a successful retry. If all parts for a message fail, see 16007.
`16007`	`ConnectionsPublishAfterRetriesFail` **ERROR**	"Failed to publish to outbound after retries" with `name`. All publish retries for a part were exhausted. The message was dropped.	Escalate: transport is persistently unavailable or rejecting messages. Check provider logs and credentials. Raising `PublishRetries` only helps if the failure is genuinely transient.
`16008`	`ConnectionsGetFileInfoFailed` **ERROR**	"Failed to get file info for relay" with `file_id`, `post_id`. FileInfo lookup failed before upload.	File may have been removed. Inspect the `error` field.
`16009`	`ConnectionsSkipOversizedFile` **WARN**	"Skipping oversized file for relay" with `file`, `size`, `max`. A file exceeded the per-connection max size and was skipped.	Expected when limits are enforced. Raise the connection's max-file-siz if intentional. The post is still relayed without the attachment.

Code	Name / Level	Log message and context	Troubleshooting
`16010`	`ConnectionsDownloadFileFailed` **ERROR**	"Failed to download file for relay" with `file_id`, `file`. Failed to download the local file bytes from Mattermost before upload.	Check Mattermost file storage health (S3, local disk, etc).
`16011`	`ConnectionsOutboundFileFiltered` **INFO**	"Outbound file filtered by policy" with `file`, `conn`. File was filtered out by the outbound extension filter.	Expected when filters are configured. Adjust the outbound filter if unintentional.
`16012`	`ConnectionsFileSemaphoreFull` **WARN**	"File semaphore full, skipping file upload" with `file`, `conn`. The file transfer concurrency semaphore was full.	Raise the file transfer concurrency setting, or investigate object store slowness. The post is sti relayed without this attachment.
`16013`	`ConnectionsUploadFileFailed` **ERROR**	"Failed to upload file" with `key`, `conn`. Upload to JetStream Object Store or Azure Blob failed.	Check provider logs and credentials. The referenced file will not arrive on the receiving side; the post envelope still goes through.

sync_user.go (17000-17999)

Emitted from the synthetic relay user subsystem, which creates local Mattermost accounts to represent remote authors when `UsernameLookup` does not find a match.

Code	Name / Level	Log message and context	Troubleshooting
`17000`	`SyncUserTruncatedUsername` **WARN**	"Truncated sync username to fit limit" with `original`, `munged`. A remote username exceeded the Mattermost 64-char username limit and was shortened.	Normal for very long remote usernames. The sync user is still created under the truncated name. Users with the same truncated prefix will collide; consider naming conventions if this happens often.
`17001`	`SyncUserLookupFallback` **DEBUG**	"Username lookup did not find local user, falling back to sync user" with `username`, `conn`. The `UsernameLookup` setting is enabled but no matching local user was found.	Expected when the remote username is not a local user. A sync user is created and used instead.
`17002`	`SyncUserAddToTeamFailed` **WARN**	"Failed to add sync user to team" with `user_id`, `team_id`. Failed to add the new sync user to the target team.	Check team membership limits and that the team allows the sync user's email domain. The post will still attempt to proceed but may not appear until the user is a team member.
`17003`	`SyncUserAddToChannelFailed` **WARN**	"Failed to add sync user to channel" with `user_id`, `channel_id`. Failed to add the sync user to the target channel.	Check channel permissions and whether the channel is private or restricted. The post may not be created until the user is a channel member.

azure_blob_provider.go (18000-18999)

Emitted from the Azure Blob WAL provider. This provider batches outgoing envelopes into a write-ahead log on local disk, uploads them as blobs, stages deferred file attachments via a companion file, and uses lease-based locking so only one receiver per container processes a given segment. Most codes relate to WAL rotation, recovery, or lock contention.

Code	Name / Level	Log message and context
`18000`	`AzureBlobContainerCreateRetry` **WARN**	"Azure Blob: transient container create err retrying" with `container`, `attempt`, `delay`. Container creation was retried o startup.
`18001`	`AzureBlobWALInTempStorage` **WARN**	"Azure Blob provider: WAL directory is in t storage and may not survive container restarts" with `wal_dir`. The WAL directo resolved to `$TMPDIR` because no persist path is configured.
`18002`	`AzureBlobWALDirFsyncFailed` **WARN**	"Azure Blob: WAL dir fsync failed" with `d` Fsync of the WAL directory failed.
`18003`	`AzureBlobMarshalPendingFailed` **WARN**	"Azure Blob: failed to marshal pending file Could not serialize pending file refs when preparing a WAL flush.
`18004`	`AzureBlobWriteCompanionFailed` **ERROR**	"Azure Blob: failed to write companion files.json" with `path`. The companion file tracks deferred file uploads for a WAL segment; writing it failed.

Code	Name / Level	Log message and context
`18005`	`AzureBlobShutdownLeftWALFiles` **WARN**	"Azure Blob: shutdown left WAL files for recovery" with `count`, `wal_dir`. At shutdown, this many WAL files had not ye been uploaded.
`18006`	`AzureBlobWALFsyncRotationFailed` **WARN**	"Azure Blob: WAL fsync failed during rotat with `path`. Fsync during WAL segment rotation failed.
`18007`	`AzureBlobWALCloseRotationFailed` **ERROR**	"Azure Blob: WAL close failed during rotat skipping upload" with `path`. The close-o rotate failed; the segment is skipped for th rotation cycle.
`18008`	`AzureBlobWALUploadFailed` **ERROR**	"Azure Blob: WAL upload failed, leaving fo recovery" with `path`. Uploading a rotate WAL segment to Azure Blob failed.
`18009`	`AzureBlobDeleteWALFailed` **WARN**	"Azure Blob: failed to delete WAL after upl with `path`. Segment uploaded successfu but local delete failed.
`18010`	`AzureBlobMarshalFailedRefsFailed` **ERROR**	"Azure Blob: failed to marshal failed refs fo companion rewrite" with `path`, `count`. Internal marshal error during companion rewrite.
`18011`	`AzureBlobRewriteCompanionFailed` **ERROR**	"Azure Blob: failed to rewrite companion files.json with failed refs" with `path`, `co` Could not rewrite the companion with the remaining failed refs after a partial upload

Code	Name / Level	Log message and context
`18012`	`AzureBlobDeleteCompanionFailed` **WARN**	"Azure Blob: failed to delete companion files.json" with `path`. All deferred upload completed but deleting the companion fail
`18013`	`AzureBlobDeferredFileFetchFailed` **ERROR**	"Azure Blob: deferred file fetch failed" with `file_id`, `post_id`. Failed to re-read a local Mattermost file for deferred upload.
`18014`	`AzureBlobDeferredFileUploadFailed` **ERROR**	"Azure Blob: deferred file upload failed" wi `file_id`, `post_id`. Failed to upload a deferred file attachment to the Azure Blob container.
`18015`	`AzureBlobShutdownMarshalResidualFailed` **ERROR**	"Azure Blob: shutdown: failed to marshal residual pending files" with `count`. At shutdown, failed to serialize residual pend state.
`18016`	`AzureBlobShutdownPersistResidualFailed` **ERROR**	"Azure Blob: shutdown: failed to persist residual pending files" with `count`, `pat` shutdown, failed to write the residual state to disk.
`18017`	`AzureBlobShutdownPersistedResidual` **WARN**	"Azure Blob: shutdown persisted residual pending files for recovery" with `count`, `path`. Residual pending state was successfully persisted for next startup.
`18018`	`AzureBlobListFailed` **ERROR**	"Azure Blob: list failed" with `container`. receiver could not list blobs in the WAL container.
`18019`	`AzureBlobDeleteRetryFailed` **WARN**	"Azure Blob: delete retry failed (marker present)" with `blob`. Retry of a blob dele (after processed-marker write) failed.

Code	Name / Level	Log message and context
`18020`	`AzureBlobDownloadFailed` **WARN**	"Azure Blob: download failed" with `blob` Failed to download a WAL segment blob during receiver poll.
`18021`	`AzureBlobHandlerError` **WARN**	"Azure Blob: handler error, will retry blob" `blob`. The downstream inbound handler returned an error for this segment.
`18022`	`AzureBlobDeleteFailed` **WARN**	"Azure Blob: delete failed" with `blob`. Fa to delete a WAL segment after successful processing.
`18023`	`AzureBlobWriteProcessedMarkerFailed` **WARN**	"Azure Blob: failed to write processed mar with `blob`. Could not write the "processe marker that prevents reprocessing.
`18024`	`AzureBlobClearProcessedMarkerFailed` **WARN**	"Azure Blob: failed to clear processed mar with `blob`. Cleanup of a stale processed marker failed.
`18025`	`AzureBlobLockGetFailed` **WARN**	"Azure Blob: lock get failed" with `blob`. Failed to read the receiver lock blob.
`18026`	`AzureBlobLockTokenGenFailed` **WARN**	"Azure Blob: lock token generation failed" `blob`. Failed to generate a random lock token (crypto rand failure).
`18027`	`AzureBlobLockSetFailed` **WARN**	"Azure Blob: lock set failed" with `blob`. F to write the receiver lock blob.

Code	Name / Level	Log message and context
`18028`	`AzureBlobCorruptLockReclaimFailed` **WARN**	"Azure Blob: corrupt lock reclaim failed" w `blob`. Tried to reclaim a corrupt (unparseable) lock blob.
`18029`	`AzureBlobStaleLockReclaimFailed` **WARN**	"Azure Blob: stale lock reclaim failed" with `blob`. Tried to reclaim a stale lock (owne died without releasing) and failed to overw
`18030`	`AzureBlobLockReleaseFailed` **WARN**	"Azure Blob: lock release failed" with `blo` Failed to release the receiver lock during shutdown.
`18031`	`AzureBlobLockReleaseGetFailed` **WARN**	"Azure Blob: lock release get failed" with `blob`. Failed to read the lock blob during release to verify ownership.
`18032`	`AzureBlobLockReleaseCorrupt` **WARN**	"Azure Blob: lock release saw corrupt valu leaving" with `blob`. At release, the lock b contents were corrupt and were left for rec
`18033`	`AzureBlobLockReleaseReclaimedByOther` **WARN**	"Azure Blob: skipping release, lock reclain by another node" with `blob`, `current_node`. Another instance reclair the lock before this one released it.
`18034`	`AzureBlobLockReleaseFailed2` **WARN**	"Azure Blob: lock release failed" with `blo` Second-stage lock release (delete of the l blob) failed.
`18035`	`AzureBlobWALRecoveryScanRootFailed` **WARN**	"Azure Blob: WAL recovery: failed to scan with `root`. Startup recovery failed to sca root of the WAL directory.
`18036`	`AzureBlobWALRecoveryScanDirFailed` **WARN**	"Azure Blob: WAL recovery: failed to scan directory" with `dir`. Failed to scan a WA subdirectory during recovery.

Code	Name / Level	Log message and context
`18037`	`AzureBlobWALRecoverySkipUnrecognized` **WARN**	"Azure Blob: WAL recovery: skipping unrecognized file" with `path`. Recovery skipped a file with an unknown name patte
`18038`	`AzureBlobWALRecoveryUploadFailed` **ERROR**	"Azure Blob: WAL recovery upload failed" `path`. Recovery failed to upload a leftov WAL segment.
`18039`	`AzureBlobWALRecoveryDeleteFailed` **WARN**	"Azure Blob: WAL recovery delete failed" v `path`. After recovery upload, the local segment delete failed.
`18040`	`AzureBlobWALRecoveryReadCompanionFail` **WARN**	"Azure Blob: WAL recovery: failed to read companion file" with `path`. Recovery fail read a segment's companion file.
`18041`	`AzureBlobWALRecoveryMalformedCompanion` **WARN**	"Azure Blob: WAL recovery: malformed companion file" with `path`. A companion exists but is unparseable.
`18042`	`AzureBlobWALRecoveryMarshalRemaining` **WARN**	"Azure Blob: WAL recovery: failed to mars remaining refs" with `path`, `count`. Inte error during recovery companion rewrite.
`18043`	`AzureBlobWALRecoveryRewriteCompanion` **WARN**	"Azure Blob: WAL recovery: failed to rewri companion file" with `path`, `count`. Cou not rewrite a recovered companion file.
`18044`	`AzureBlobFileListFailed` **ERROR**	"Azure Blob: file list failed" with `containe` Failed to list deferred file blobs during rece polling.
`18045`	`AzureBlobFileDeleteRetryFailed` **WARN**	"Azure Blob: file delete retry failed (marke present)" with `blob`. Retry of deferred fil blob delete failed.
`18046`	`AzureBlobFileDownloadFailed` **WARN**	"Azure Blob: file download failed" with `bl` Failed to download a deferred file blob.

Code	Name / Level	Log message and context
`18047`	`AzureBlobFileHandlerError` **WARN**	"Azure Blob: file handler error" with `blob` The file handler rejected a downloaded deferred file.
`18048`	`AzureBlobFileDeleteFailed` **WARN**	"Azure Blob: file delete failed" with `blob` Failed to delete a deferred file blob after successful processing.
`18049`	`AzureBlobSPCredentialFailed` **ERROR**	"Azure Blob: service principal credential fa with `tenant_id`, `client_id`, and a sanitized `error`. `azidentity.NewClientSecretCredent` rejected the supplied Service Principal credentials at provider construction time.

azure_provider.go (19000-19999)

Emitted from the Azure Queue provider, which uses an Azure Storage Queue for envelopes and a paired Azure Blob container for file attachments.

Code	Name / Level	Log message and context	T
`19000`	`AzureQueueCreateQueueFailed` **WARN**	"Azure Queue: could not create queue (may already exist)" with `queue`. Idempotent startup create returned an error.	U e c q p
`19001`	`AzureQueueCreateContainerFail` **WARN**	"Azure Blob: could not create container (may already exist)" with `container`. Idempotent blob container create returned an error.	S

Code	Name / Level	Log message and context	T
`19002`	`AzureQueueDequeueFailed` **ERROR**	"Azure Queue dequeue failed" with `queue`. Failed to dequeue a batch of messages.	C P tr re c
`19003`	`AzureQueueDecodeFailed` **ERROR**	"Azure Queue: failed to decode message" with `queue`. A dequeued message body was not a valid Cross Guard envelope.	C s d w
`19004`	`AzureQueueHandlerRetry` **WARN**	"Azure Queue: handler returned error, message will retry" with `queue`. The downstream inbound handler returned an error; the message is left on the queue for the next visibility timeout.	C S m c m
`19005`	`AzureQueueDeleteProcessedFail` **WARN**	"Azure Queue: failed to delete processed message" with `queue`. Message was processed but the delete call failed.	M lo a id d
`19006`	`AzureQueueBlobListFailed` **ERROR**	"Azure Blob list failed" with `container`. Failed to list deferred file blobs in the paired container.	C
`19007`	`AzureQueueBlobDownloadFailed` **WARN**	"Azure Blob: failed to download" with `blob`. Failed to download a deferred file blob.	C n
`19008`	`AzureQueueBlobHandlerError` **WARN**	"Azure Blob: handler returned error" with `blob`. File handler rejected a downloaded blob.	C e
`19009`	`AzureQueueBlobDeleteFailed` **WARN**	"Azure Blob: failed to delete after processing" with `blob`. Failed to delete a blob after successful processing.	C

Code	Name / Level	Log message and context	T
`19010`	`AzureQueueSPCredentialFailed` **ERROR**	"Azure Queue: service principal credential failed" with `tenant_id`, `client_id`, and a sanitized `error`. `azidentity.NewClientSecretCredential` rejected the supplied Service Principal credentials at provider construction time.	V F ` e s fu ` o

nats_provider.go (20000-20999)

Emitted from the NATS transport provider and its JetStream Object Store file handling.

Code	Name / Level	Log message and context	Troubleshooting
`20000`	`NATSDownloadFileFailed` **ERROR**	"Failed to download file from object store" with `key`. JetStream Object Store download failed.	Check JetStream health on the NATS broker (`nats stream ls`), confirm the object store bucket exists, and check the `key` has not been pruned.
`20001`	`NATSFileHandlerError` **WARN**	"File handler returned error" with `key`. Downstream file handler rejected a downloaded object.	Correlate with the adjacent `15041`-`15045` inbound file error.
`20002`	`NATSDisconnected` **WARN**	NATS client disconnect callback with `name`. Log message is generated by the NATS client library.	Transient if followed by 20003 soon after. Persistent disconnects mean the broker is unreachable or the client is being forcibly kicked (auth rotation, account cap).

Code	Name / Level	Log message and context	Troubleshooting
`20003`	`NATSReconnected` **INFO**	NATS client reconnect callback with `name`. Informational.	Informational, confirms recovery after a 20002 event.

prompt.go (21000-21999)

Emitted from the inbound prompt subsystem. Prompts are interactive posts (Accept / Block buttons) shown when a new inbound connection tries to deliver to a team or channel that has not yet opted in.

Code	Name / Level	Log message and context	Troubleshooting
`21000`	`PromptGetConnPromptFailed` **ERROR**	"Failed to get connection prompt" with `team_id`, `conn`. Failed to read the stored team-level prompt state.	Inspect the KV store error.
`21001`	`PromptGetTownSquareFailed` **ERROR**	"Failed to get town-square for prompt" with `team_id`. Failed to find town-square when posting a team-level prompt.	Verify the team has a town-square channel. Rare; only occurs if the default channel was deleted, which Mattermost normally prevents.
`21002`	`PromptCreatePostFailed` **ERROR**	"Failed to create connection prompt post" with `team_id`, `conn`. Failed to post the prompt message.	Confirm the bot can post in town-square and the post payload does not violate server limits.

Code	Name / Level	Log message and context	Troubleshooting
`21003`	`PromptSavePromptFailed` **ERROR**	"Failed to save connection prompt" with `team_id`, `conn`. Failed to save the prompt state after posting.	Inspect the KV store error.
`21004`	`PromptAcceptGetPromptFailed` **ERROR**	"Failed to get connection prompt" with `team_id`, `conn`. Accept handler could not load the stored prompt state.	Inspect the KV store error. The admin may need to click Accept again after the store recovers.
`21005`	`PromptDeletePromptFailed` **ERROR**	"Failed to delete connection prompt" with `team_id`, `conn`. Failed to delete the prompt state after Accept.	Prompt may reappear on the next inbound message until the delete succeeds. Use `reset-prompt` to manually clear.
`21006`	`PromptBlockGetPromptFailed` **ERROR**	"Failed to get connection prompt" with `team_id`, `conn`. Block handler could not load state.	Inspect the KV store error.
`21007`	`PromptSetBlockedFailed` **ERROR**	"Failed to update connection prompt to blocked" with `team_id`, `conn`. Failed to mark the prompt as blocked.	Inspect the KV store error.
`21008`	`PromptGetChanPromptFailed` **ERROR**	"Failed to get channel connection prompt" with `channel_id`, `conn`. Failed to read	Inspect the KV store error.

Code	Name / Level	Log message and context	Troubleshooting
		channel-level prompt state.	
`21009`	`PromptCreateChanPostFailed` **ERROR**	"Failed to create channel connection prompt post" with `channel_id`, `conn`. Failed to post the channel-level prompt.	Confirm the bot can post in the target channel.
`21010`	`PromptSaveChanPromptFailed` **ERROR**	"Failed to save channel connection prompt" with `channel_id`, `conn`. Failed to save channel-level prompt state.	Inspect the KV store error.
`21011`	`PromptChanAcceptGetFailed` **ERROR**	"Failed to get channel connection prompt" with `channel_id`, `conn`. Channel-level Accept handler failed to load state.	Inspect the KV store error.
`21012`	`PromptDeleteChanPromptFailed` **ERROR**	"Failed to delete channel connection prompt" with `channel_id`, `conn`. Failed to delete channel-level prompt state after Accept.	Use `reset-channel-prompt` to clear manually.
`21013`	`PromptChanBlockGetFailed` **ERROR**	"Failed to get channel connection prompt" with `channel_id`, `conn`. Channel-level	Inspect the KV store error.

Code	Name / Level	Log message and context	Troubleshooting
		Block handler failed to load state.	
`21014`	`PromptSetChanBlockedFailed` **ERROR**	"Failed to update channel connection prompt to blocked" with `channel_id`, `conn`. Failed to set channel-level blocked state.	Inspect the KV store error.
`21015`	`PromptGetPostForUpdateFailed` **ERROR**	"Failed to get prompt post for update" with `post_id`. Could not load the prompt post to update its displayed message after user action.	Post may have been deleted. Non-fatal; the state is still updated.
`21016`	`PromptUpdatePostFailed` **ERROR**	"Failed to update prompt post" with `post_id`. Could not update the prompt post to reflect the user's Accept/Block action.	Check bot post permissions. Non-fatal; the prompt state change is persisted even if the visible post still shows the old buttons.

retry_dispatch.go (22000-22999)

Emitted from the retry queue that holds inbound messages whose parent post has not yet been received. A background ticker reprocesses the queue.

Code	Name / Level	Log message and context	Troubleshooting
`22000`	`RetryDispatchSucceeded` **WARN**	"Missing message: retry succeeded" with `conn`, `type`, `remote_post_id`, `attempt`. A queued message was successfully dispatched after its parent arrived.	Informational for operators but logged at WARN so it is visible. Rising `attempt` counts indicate systemic out-of-order delivery.
`22001`	`RetryDispatchStillMissing` **WARN**	"Missing message: still missing after retry" with `conn`, `type`, `remote_post_id`, `attempt`, `next_retry_in`. Parent is still missing; the message stays queued.	Expected for a few attempts during out-of-order delivery. If this repeats past `attempt` = max-retries, see 22004.
`22002`	`RetryDispatchDropMaxAge` **ERROR**	"Missing message: dropped, exceeded max age" with `conn`, `type`, `remote_post_id`, `age`. Parent never arrived within max age; message dropped.	The parent post never made it through. Check the sender's logs for the `remote_post_id` and see whether an earlier error (15xxx, 16xxx, or sender-side) blocked it. Consider raising the max-age setting if senders are legitimately slow.
`22003`	`RetryDispatchDropUnmarshal` **ERROR**	"Missing message: dropped, payload unmarshal failed on retry" with `conn`, `type`, `remote_post_id`. A	Usually indicates corruption at rest or a codec version mismatch. The envelope was valid when first received, so investigate memory or KV store corruption.

Code	Name / Level	Log message and context	Troubleshooting
		queued message failed to unmarshal on retry.	
`22004`	`RetryDispatchDropMaxRetries` **ERROR**	"Missing message: dropped after max retries" with `conn`, `type`, `remote_post_id`, `attempts`. A queued message exceeded the max retry count.	Parent cannot be resolved. Investigate whether the parent post is being filtered out (e.g., blocked prompt, inactive channel, unresolved user) rather than delayed.

store/caching.go (23000-23999)

Emitted from the in-memory cache that sits in front of the KV store on each node.

Code	Name / Level	Log message and context	Troubleshooting
`23000`	`StoreCachePublishInvalidationFailed` **WARN**	"Failed to publish cache invalidation event" with `event`, `key`. Could not publish a cluster cache invalidation message after a KV write.	In a multi-node Mattermost cluster, other nodes may serve stale data until the next TTL (default 30 seconds). Check that the Mattermost cluster plugin message bus is healthy. Single-node deployments can ignore this code.

request.go (24000-24999)

Emitted from the team admin connection request workflow. When both `RestrictToSystemAdmins` and `AllowTeamAdminRequests` are enabled, Team Admins can submit link requests that System Admins approve or deny via interactive DM buttons. These codes cover request creation, approval, denial, cancellation, and notification delivery.

Code	Name / Level	Log message and context	Troubleshooti
`24000`	`RequestGetFailed` **ERROR**	"Failed to get connection request" with `team_id`, `conn_key`. Also used during cancellation (WARN level): "Failed to get connection request for cancellation". KV store lookup for an existing pending request failed.	Check the `error` field fo the underlying KV store error. Usually transie If repeated, check KV store health. The request creatio or cancellation will be aborted.
`24002`	`RequestGetDMChannelFailed` **ERROR**	"Failed to get DM channel for connection request" with `admin_id`. Could not open a DM channel between the bot and a System Admin to deliver the request notification.	Verify the bot user exists and the admin account is activ This admin will not receive the request DM, bu others still will. all admins fail, the request ma be saved with r notification pos
`24003`	`RequestCreateDMPostFailed` **ERROR**	"Failed to create DM post for connection request" with `admin_id`. The DM channel was opened but the post with Approve/Deny buttons could not be created.	Check Mattermost server health a post creation permissions. Th admin will not see the reques If all admins fai the request will be saved but n

Code	Name / Level	Log message and context	Troubleshooti
			one can act on from the UI.
`24004`	`RequestSaveFailed` **ERROR**	"Failed to save connection request" with `team_id`, `conn_key`. Could not persist the pending request to the KV store after DM posts were created.	The DM posts will be cleaned up automaticall Inspect the KV store error. The Team Admin wi see a generic "failed to save" message. Retry the command.
`24005`	`RequestApproveGetFailed` **ERROR**	"Failed to get connection request for approve" with `team_id`, `conn_key`. When a System Admin clicks Approve, the handler could not load the pending request from KV store.	Check KV store health. The admin will see "Failed to check request status." in the post actio response. The request remain pending and ca be retried.
`24006`	`RequestApproveExecFailed` **ERROR**	"Failed to execute approved connection request" with `team_id`, `conn_key`. The request was approved but the underlying `initTeamForCrossGuard` call failed.	Inspect the nested error message. Common cause team deleted, connection con changed, KV store error. The request remain pending. Fix the underlying issu and have the admin click Approve again.

Code	Name / Level	Log message and context	Troubleshooti
`24007`	`RequestApproveDeleteFailed` **ERROR**	"Failed to delete approved connection request" with `team_id`, `conn_key`. The connection was linked successfully but the pending request record could not be cleaned up.	Non-critical: the connection is active. The stal request record will be detected as "no longer active" if someone clicks Approve/Deny again. No actio needed unless persists.
`24008`	`RequestApproveNotifyFailed` **ERROR**	"Failed to get DM channel for requester notification" or "Failed to notify requester" with `requester_id`. Could not send the approval/denial notification DM to the requesting Team Admin.	Check that the requester account still exists and the bot can open a DM. The connection acti (approve/deny) was still completed successfully; or the notification was lost.
`24009`	`RequestDenyDialogFailed` **ERROR**	"Failed to open deny dialog" with `team_id`, `conn_key`. The Deny button was clicked but the interactive dialog (for entering a reason) could not be opened.	Check Mattermost server health. The request remains pendin The admin can try clicking Den again or use th Approve button instead.
`24010`	`RequestDenyGetFailed` **ERROR**	"Failed to get connection request for deny submit" with `team_id`,	Check KV store health. The

Code	Name / Level	Log message and context	Troubleshooti
		`conn_key`. After the deny dialog was submitted, the handler could not load the pending request.	denial was not processed. The admin can retry from the origina DM buttons.
`24011`	`RequestDenyDeleteFailed` **ERROR**	"Failed to delete denied connection request" with `team_id`, `conn_key`. Also used during cancellation (WARN level): "Failed to delete connection request during cancellation". The deny or cancel succeeded logically but the KV record was not removed.	Non-critical: same as 24007 The stale recor will be detectec as "no longer active" on next interaction.
`24012`	`RequestDenyNotifyFailed` **ERROR**	Reserved for denial notification failures. Currently shares the `RequestApproveNotifyFailed` code path (24008) for requester notification.	See 24008. Thi code is allocate for future use if deny notificatio need a distinct code path.
`24013`	`RequestUpdatePostFailed` **WARN**	"Failed to get request post for update" or "Failed to update request post" with `post_id`. After approval, denial, or cancellation, the handler could not update one of the admin DM posts to remove the Approve/Deny buttons.	Non-critical: the request action was still processed. The admin may see stale buttons, b clicking them w show "This request is no longer active." Post may have been deleted manually.
`24014`	`RequestNoSystemAdmins` **WARN**	"No system admins available to review connection request" with `team_id`, `conn_key`. The Team Admin submitted a request	Ensure at least one System Admin account exists and is

Code	Name / Level	Log message and context	Troubleshooti
		but the system has no admin users to notify.	active. The request was no created. The Team Admin sees "no syste admins availab to review your request."
`24015`	`RequestConnRemovedFromConfig` **WARN**	"Connection removed from config while request was pending" with `team_id`, `conn_key`. A System Admin clicked Approve but the connection no longer exists in plugin configuration.	The request is automatically cancelled and the requester is notified. The connection mus be re-added to the configuratio before a new request can be submitted.
`24016`	`RequestConfirmDMFailed` **WARN**	"Failed to get DM channel for requester confirmation" or "Failed to send requester confirmation DM" with `user_id`. After DM notifications were sent to System Admins, the confirmation DM to the requesting Team Admin could not be delivered.	Non-critical: the request was created and admins were notified. Only th requester's confirmation message was lost. Check that the requester account is activ and the bot can open a DM.

Code	Name / Level	Log message and context	Troubleshooti
`24017`	`RequestNoAdminsNotified` **ERROR**	"Failed to notify any system admin about connection request" with `team_id`, `conn_key`. DM delivery failed for every System Admin. The reserved request is cleaned up automatically.	Ensure at least one System Admin account active and the bot can open D channels. The Team Admin sees "failed to notify any syste admin about yc request." Retry after fixing adm accounts.

channel_request.go (25000-25999)

Emitted from the channel admin connection request workflow. When both `RestrictToSystemAdmins` and `AllowChannelAdminRequests` are enabled, Channel Admins can submit channel-level link requests that Team Admins approve or deny via interactive DM buttons. These codes cover request creation, approval, denial, cancellation, and notification delivery.

Code	Name / Level	Log message and context	Troublesho
`25000`	`ChanRequestGetFailed` **ERROR**	"Failed to get channel connection request" with `channel_id`, `conn_key`. Also used during cancellation (WARN level): "Failed to get channel connection request for cancellation". KV store lookup for an existing pending channel request failed.	Check the `e` for the under store error. U transient. If r check KV sto The request cancellation aborted.
`25002`	`ChanRequestGetDMChannelFailed` **ERROR**	"Failed to get DM channel for channel connection request" with `admin_id`. Could not open a DM channel between	Verify the bo exists and th Admin accou This admin w

Code	Name / Level	Log message and context	Troubleshoo
		the bot and a Team Admin to deliver the request notification.	receive the r but others st
`25003`	`ChanRequestCreateDMPostFailed` **ERROR**	"Failed to create DM post for channel connection request" with `admin_id`. The DM channel was opened but the post with Approve/Deny buttons could not be created.	Check Matte server health creation perr This admin w the request.
`25004`	`ChanRequestSaveFailed` **ERROR**	"Failed to save channel connection request" with `channel_id`, `conn_key`. Could not persist the pending channel request to the KV store. Also used (WARN level) when updating the stored request with post IDs fails.	Inspect the K error. The Ch Admin will se "failed to sav message. Re command.
`25005`	`ChanRequestApproveGetFailed` **ERROR**	"Failed to get channel connection request for approve" with `channel_id`, `conn_key`. Also used when verifying team connections during approval: "Failed to get team connections during channel request approve" with `team_id`.	Check KV st The admin w error in the p response. Th remains pen can be retrie
`25006`	`ChanRequestApproveExecFailed` **ERROR**	"Failed to execute approved channel connection request" with `channel_id`, `conn_key`. The request was approved but the underlying `initChannelForCrossGuard` call failed.	Inspect the n message. Co causes: char deleted, tean connection re KV store erro underlying is have the adr Approve aga

Code	Name / Level	Log message and context	Troubleshoo
`25007`	`ChanRequestApproveDeleteFailed` **ERROR**	"Failed to delete channel connection request during approve" with `channel_id`, `conn_key`. The pending request record could not be removed after approval started (delete-first pattern).	The request be aborted. T sees "Failed request. Plea again." Retry checking KV health.
`25008`	`ChanRequestApproveNotifyFailed` **ERROR**	Reserved for approval notification failures. Currently uses the shared `notifyRequester` and `updateRequestPosts` helpers.	See the `updateRequ` and `notifyRequ` error handlin critical if the was linked su
`25009`	`ChanRequestDenyDialogFailed` **ERROR**	"Failed to open channel deny dialog" with `channel_id`, `conn_key`. The Deny button was clicked but the interactive dialog could not be opened.	Check Matte server health request rema pending. The try clicking D
`25010`	`ChanRequestDenyGetFailed` **ERROR**	"Failed to get channel connection request for deny" or "Failed to get channel connection request for deny submit" with `channel_id`, `conn_key`. The handler could not load the pending channel request during denial.	Check KV st The denial w processed. T can retry fron original DM b
`25011`	`ChanRequestDenyDeleteFailed` **ERROR**	"Failed to delete denied channel connection request" with `channel_id`, `conn_key`. Also used during cancellation (WARN level): "Failed to delete channel connection request during cancellation".	Non-critical: record will be as "no longe next interacti

Code	Name / Level	Log message and context	Troubleshoo
`25012`	`ChanRequestDenyNotifyFailed` **ERROR**	Reserved for denial notification failures. Currently shares the `notifyRequester` and `updateRequestPosts` code paths.	See 25008. T allocated for if deny notifi need a distin path.
`25013`	`ChanRequestUpdatePostFailed` **WARN**	Reserved for post update failures during channel request lifecycle. Currently uses the shared `updateRequestPosts` helper which logs via `RequestUpdatePostFailed` (24013).	Non-critical: action was s processed. T may see stal but clicking t show "This r longer active
`25014`	`ChanRequestNoTeamAdmins` **WARN**	"No team admins available to review channel connection request" with `channel_id`, `team_id`, `conn_key`. The Channel Admin submitted a request but no Team Admins exist for this team.	Ensure at lea Team Admin the team. Th was not crea Channel Adm "no team adm available to r request."
`25015`	`ChanRequestConnRemovedFromConfig` **WARN**	"Connection removed from config while channel request was pending" with `channel_id`, `conn_key`. A Team Admin clicked Approve but the connection no longer exists in plugin configuration.	The request automatically and the requ notified. The must be re-a configuration new request submitted.

Code	Name / Level	Log message and context	Troubleshoo
`25016`	`ChanRequestConfirmDMFailed` **WARN**	"Failed to get DM channel for channel requester confirmation" or "Failed to send channel requester confirmation DM" with `user_id`. After DM notifications were sent to Team Admins, the confirmation DM to the requesting Channel Admin could not be delivered.	Non-critical: was created Admins were Only the requ confirmation was lost. Che requester ac active.
`25017`	`ChanRequestNoAdminsNotified` **ERROR**	"Failed to notify any team admin about channel connection request" with `channel_id`, `team_id`, `conn_key`. DM delivery failed for every Team Admin. The reserved request is cleaned up automatically.	Ensure at lea Team Admin active and th open DM cha Channel Adm "failed to not team admin request." Ref fixing Team A accounts.
`25018`	`ChanRequestGetTeamAdminsFailed` **ERROR**	"Failed to get team members for admin lookup" with `team_id`. The paginated team member lookup to find Team Admins failed. Also logged (WARN level) when an individual user lookup within the pagination loop fails: "Failed to get user during team admin lookup" with `user_id`.	Check Matte health. The C Admin will se "failed to look admins" erro command. If user lookups users are ski others may s notified.

azure_servicebus_provider.go (26000-26999)

Emitted from the Azure Service Bus provider, which uses an Azure Service Bus queue (PeekLock receive mode) for envelopes and an optional Azure Blob Storage sidecar container for file attachments. Service Bus send failures are returned to the caller rather than logged by the

provider; upstream retry dispatch logs those via its own error code. The code `APIAzureServiceBusTestFailed` (26006) is emitted from `api.go` but is grouped here because it reports a Service Bus connection failure.

Code	Name / Level	Log message and context	Tr
`26000`	`ServiceBusSendFailed` **WARN**	"Service Bus: could not create blob container (may already exist)" with `container`. Idempotent blob sidecar container create returned an error at provider startup. (Reserved for send-path logging; `Publish` currently returns sanitized errors to the caller rather than logging them locally.)	Us (co ex ve cre co co an in ac
`26001`	`ServiceBusReceiveFailed` **ERROR/WARN**	"Service Bus receive failed" (ERROR) with `queue`, and (WARN) "Service Bus blob list failed", "Service Bus blob: failed to download", "Service Bus blob: handler returned error", "Service Bus blob: failed to delete after processing", each with the corresponding `container` or `blob` field. Covers both the Service Bus receive loop and the blob sidecar poll.	Ch rea Se na en loc co inc fai the

Code	Name / Level	Log message and context	Tr
`26002`	`ServiceBusCompleteFailed` **WARN**	"Service Bus complete failed; message will redeliver" or "Service Bus complete failed on empty-body drop; message will redeliver" with `queue` and `message_id`. Complete (settle) call against a received message failed.	Th ex me rec au inb ide ide so su Re po ne pe
`26003`	`ServiceBusAbandonFailed` **WARN**	"Service Bus abandon failed" with `queue`. Abandon call after a handler error failed; the lock will expire instead of being released explicitly.	No me rec loc Inv wit ev

Code	Name / Level	Log message and context	Tr
`26004`	`ServiceBusRedelivery` **WARN**	"Service Bus message redelivered" with `queue`, `delivery_count`, `message_id`. A received message has a `DeliveryCount > 1`, meaning Service Bus has redelivered it at least once.	Ex tra fai rat rec po Se mc to qu qu `Ma` is Az `se` `me` the
`26005`	`ServiceBusMalformedBody` **WARN**	"Service Bus message has empty body; completing to drop" with `queue`, `message_id`. A received message had a zero-length body. The plugin completes (drops) it to avoid an infinite redelivery loop.	Ca ID Us pro the se em

Code	Name / Level	Log message and context	Tr
`26006`	`APIAzureServiceBusTestFailed` **ERROR**	"Azure Service Bus connection test failed" with `error` (sanitized). Emitted from `handleTestAzureServiceBusConnection` when the admin "Test Connection" button cannot peek the target queue.	Th alr SA Ty qu (pr Az AF un the SA pe un (ne
`26007`	`ServiceBusSPCredentialFailed` **ERROR**	"Service Bus: service principal credential failed" with `tenant_id`, `client_id`, and a sanitized `error`. `azidentity.NewClientSecretCredential` rejected the supplied Service Principal credentials at provider construction time.	Ve (G `c` tha `c` no fie cre

Back to Overview